Module Code & Module Title

CS6PO5NT-Final Year Project

Assessment Weightage & Type

Final Year Project Interim Report

Year and Semester

2025-26 Autumn

Student Name: Nisha Subedi

London Met ID: 23049293

College ID: np05cp4a230236@iic.edu.np

Assignment Submission Date: 26/12/2025

Internal Supervisor: Sujan Subedi

External Supervisor: Utsav Dhungana

Title: AI-Powered Crop Disease Detection and Recommendation System

I confirm that I understand my coursework needs to be submitted online via Google Classroom under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Table of Contents

Table of Figure

Table of Tables

1. Introduction

1.2. Introduction to the FYP

As being a student of the BSc (Hons) Computing program at Itahari International College (IIC), Final Year Project (FYP) is the final evidence of my academic experience during which, I would be able to demonstrate the extent of knowledge I have gained in terms of technical knowledge and skills, creativity and problem-solving I acquired during my time at IIC. To support a solution to a problem in the real-world situation in the field of agriculture my fyp project has independently brainstormed the concept of an AI-based Crop Disease Detection System. Delayed diagnosis, losses and reliance by farmers on experts to diagnose plant diseases are some of the challenges that farmers usually encounter. After realizing this, I chose to create a system that can harness the power of the artificial intelligence and computer vision to identify diseases at an early stage and with precision to give a timely advice to avoid harm. It is a personal project which will include the training of AI models, software development, database management and web/mobile app design as a practical and innovative solution. The project aims to produce a user friendly application, be insightful in real-time and also to make the system scalable, maintainable and efficient. On the whole, this FYP is indicative of my originality, technical skill and aptitude to translate a computing concept into an actual solution with practical significance to satisfy the demands of my BSc (Hons) degree Computing at IIC.

1.3. Introduction to the AI-Powered Crop Disease Detection and Recommendation System

Figure 1:AI-Powered Crop Disease Detection and Recommendation System

Agriculture is still among the key pillars that have helped global food security, economic stability and human livelihood. In nations where agriculture is the main income source, even a small change in crop diseases may cause a domino effect on the crop productivity causing food shortage, varied food availability and economic stress amongst farming societies. Crop diseases are one of the challenges that have been experienced in this industry but which have caused a very destructive danger with no voice. Diseases in plants usually start unobtrusively through leaves, stems and soil with no visible symptoms. By the time the symptoms develop to a stage where farmers can take action, it could be too late and much of the crop could be spoiled hence leading

to irredeemable losses in production. This invisibility at early stages renders timely and correct detection a very imperative aspect of disease management and general sustainability of agriculture.

The traditional way of identifying the disease still continues to be used by many farmers and involves physical examination or informal arrangements with other farmers or local specialists. Though they are ancient practices that have been practiced across generations, they are not precise and consistent. The visual symptoms of various diseases are similar and thus, manual identification is likely to be subject to error. The situation is also worse in rural and remote agricultural areas where there are limited access to modern diagnostic support, as well as, the availability of plant pathology professionals. Farmers might find themselves traveling long distances to consult professionals and by the time the farmers are assisted, the disease may have spread to cover a whole field. Not only it translates into the loss of money at a personal level, but also into reduced agricultural production, which can affect food supplies in the area and food prices.

The other significant drawback of the traditional identification is that it requires human interpretation. There are various factors which influence the health of a plant such as the temperature, the nutrition of the soil, humidity, pests, or even fungal activity most of which would be impossible without scientific equipment to tell the difference. An illness can be similar to a nutrient deficiency, pest attack or biological stress which drives farmers to futile treatment methods, spending resources and time. The improper use of treatment goes further to accelerate the deterioration of crops and in some instances leads to overuse of the chemicals in pesticides which is dangerous to the quality of soil and health of the consumers. Thus, the agricultural population is becoming more and more specific when requiring a scientific, rapid, and reliable solution which will help eliminate the uncertainty and time-consuming aspect of detecting the disease.

Image classification has become a reality due to the development of Artificial Intelligence and with the advent of the technology, a new way of managing crop diseases has become possible. Within the image of leaves, AI based systems are able to identify and report the disease accurately and also provide treatment that can be taken. AI is scalable, consistent and not influenced by human error compared to manual observation. We will be able to empower farmers to make their decisions more quickly, decrease losses of crops and enhance the large-scale agricultural production by providing them with instant diagnostic of the disease. That is the vision on which this project is based to achieve a smarter and data assisted ecosystem into the transformation of traditional agricultural practices, where timely information translates to healthier farms and stronger communities.

1.3.1. Problem Scenario

One of the long term challenges that have affected agriculture in all parts of the world is crop diseases especially in areas where agriculture is the major economic activity and livelihood. Thousands of acres of agricultural land fall victim to fungal, bacterial and viral infections of plants that are not even noticed before the damage is too late to repair. The very essence of the crop diseases is unpredictable in nature because the disease may be caused by climatic changes, soil imbalance, ineffective irrigation methods, insect vectors and also the microbial infections. All these factors lead to a complex agricultural environment where early detection is not only useful to achieve crop health and yield but necessary.

Nonetheless, the actual problem is presented in the fact that the gap between the occurrence of the disease and its recognition is an issue. Most agricultural societies particularly in the third world countries are highly dependent on physical observation to detect the signs of illnesses like discoloration of leaves, spots, withering and molds. Regrettably, such symptoms are noticeable when the infection has already attained a high level. In addition, most plant diseases have a similar outward appearance that a farmer cannot distinguish with the help of a scientific device or an expert. Such uncertainty tends to give the wrong assumptions, inadequate treatment decisions and tardiness in taking preventative actions all of which result in promoting crop degradation and monetary waste.

Accessibility to agricultural experts is also another important issue of the problem. The trained consultants are certified plant pathologists, agriculture officers but the population of farmers is huge. A large number of farmers are located too far away in agricultural centers and do not have the necessary ones or resources to contact experts as quickly as possible. Consultations can be time-consuming even with the option of consultation, they can spend days, weeks and in most cases incur extra expenses in terms of transport, labor and external diagnosis. A combination of these hindrances contributes to massive loss of crop, low productivity and high dependence on external markets crop products which eventually impacts on national food security and economic stability.

1.3.2. The Project as a Solution

The AI-Powered Crop Disease Detection and Recommendation System is the solution suggested in this project which is aimed at serving the modern agriculture sector with the help of intelligent diagnostics and advisory recommendations. In contrast to the traditional approach where the information is gathered manually and the expert is available to analyze the information

and identify the crop disease, the given system uses machine learning and image recognition to identify the disease in the crop based on the leaf image in real-time. The proposed platform with Convolutional Neural Networks (CNN), MERN technology and an analytical dashboard would help decrease the detection time, enhance the accuracy of treatment and create a stable agricultural decision-support ecosystem.

Such a system gives farmers the opportunity to post a picture of an infected leaf via a convenient web interface. The trained AI model processes the picture, detects patterns of diseases and gives a comprehensive result which will contain disease name, the confidence score, level of severity and recommended treatment options. Farmers are now informed in real-time with the power to make timely and effective recovery measures as opposed to waiting until an agricultural specialist makes the assumptions that determine steps to be taken.

i. Core Solution Capabilities

- Disease detection through CNN based deep learning model.
- Output of disease within seconds following image uploads.
- Fertilizer, pesticide and organic treatment suggestions.
- Disease trends and frequency patterns visualized with the use of charts.
- All predictions have a history, which is guaranteed by cloud-based storage.
- Is a Progressive Web App (PWA) that can work on mobile and low-bandwidth environments.
- Admin dashboard to add, manage or update disease datasets.

The best thing about this system is that it is not only able to diagnose but also guide, monitor and educate. It also does not end at the identification farmers are made structured recommendations to treatment that are in line with sustainable agricultural practices. Past performance is also recorded in the system, which makes farmers to keep track of the performance and observe the patterns of recurrence over time.

ii. How This System Solves Existing Challenges

Table 1 : Table of Traditional Problem and AI-Based Solution

S.N	Traditional Problem	AI-Based Solution
i.	Slow and inaccurate manual identification.	Instant disease prediction powered by CNN.
ii.	Lack of expert access in rural areas.	Accessible from any device with internet.
iii.	Confusion between similar disease symptoms.	High accuracy pattern recognition model.

iv.	No documentation for case history.	Auto saved prediction logs and reports.
v.	Overuse of pesticides.	Balanced treatment and prevention suggestions.

iii. Direct Benefits to Farmers

- Faster diagnosis → early treatment → reduced crop loss
- Increased yield and revenue due to timely intervention
- Lower dependence on physical experts and middlemen
- Better planning for future farming cycles through history analytics
- Improved soil and crop health with informed chemical usage

Overall, this is a project that converts conventional farming to smart farming, which is a combination of technology and field viability. The AI-powered system can serve as a digital agricultural assistant through which with a touch of a button, farmers can obtain accurate, empowering and more direct and timely information, able to work round the clock, scale easily and enable farmers to make reliable decisions. This system has the potential to make agriculture efficient, intelligent and resilient with the ability to diversify its crops, types of diseases and health monitoring.

2. Aim and Objectives

The primary aim of this Final Year Project is to create a full-fledged end-to-end AI-based crop disease detection and advisory system that will be able to interpret the images of plant leaves correctly and provide a diagnosis and a treatment prescription in real time. The system aims at assisting the agricultural sector by decreasing reliance on the manual diagnosis, reducing the financial loss as a result of disease outbreaks, and facilitating the farmer in making better farming decisions with the help of the digital solution. This project will fill the gap between the early detection of diseases and taking corrective measures on time by incorporating Artificial Intelligence, deep learning, REST API communication, and dashboard analytics based on MERN.

In its essence, the project is aimed at enhancing agricultural productivity, sustainability and accuracy of decisions. The traditional agriculture mostly relies on experience-based recognition that is liable to misunderstanding and slowness. Compared to it, this AI system facilitates the

prediction of the disease through visual identification by comparison with a learned set of symptoms so that even unprofessional users can identify the problem in a short period of time and with high accuracy. With the additions of history tracking, notification, and visual analytics, this project is not only a detection platform, but an entire agricultural intelligence platform.

2.1. Aim of The Project

The main aim of the project is to create and innovate an AI-powered web application that can detect the disease of crops based on uploaded photos of leaves, give correct treatment information and give analytical data to help farmers avoid crop loss and enhance agricultural production.

2.2. Objectives of The Project

The objectives of the project are given below:

i. Technical Development Objectives

- To train a CNN-based deep learning model using an agricultural dataset for disease classification.
- To implement a Flask API microservice that processes uploaded images and returns prediction results.
- To develop a MERN-based web interface where users can upload images and view results.
- To store prediction logs, user records, and disease data in a scalable MongoDB database.
- To integrate analytical graphs using Chart.js/Recharts for trend visualization.
- To develop a Progressive Web Application (PWA)-based UI for mobile accessibility.
- To support history tracking, notification alerts, and offline caching for usability.

ii. Functional Output Objectives

- To automatically detect diseases with high accuracy and confidence percentage.
- To provide remedy solutions including:
  i. Organic remedies
  ii. Fungicides & pesticides
  iii. Fertilizer recommendation & dosage
- To store past predictions for monitoring disease patterns over time.

- To allow admin users to update disease categories, retrain the AI model & manage users.
- To generate region-wise disease frequency reports and usage stats for agricultural study.

iii. End-User Benefit Objectives

- Minimize crop damage by diagnosing diseases at early stages.
- Reduce farmers' dependency on external consultancy for basic diagnosis.
- Increase crop production through fast intervention and guided treatment.
- Improve resource management by reducing wasted pesticide usage.
- Enable farmers to digitally track plant health progression across seasons.

2.3. Why This Objective Matters

With the achievement of these goals, this project will directly affect the smart agriculture transformation which will enable technology to substitute the uninformed decision-making process with the data analysis. Farmers are not obliged to wait days waiting the inspection of the experts but they get the results immediately and make sure that the treatment is provided at the appropriate time, thus avoiding the spread of diseases and loss of money. This system can be scaled, be future ready and able to move to other types of crops and monitor agriculture at a national level, thanks to the combination of AI and dashboard analytics.

3. Excepted Outcomes and Deliverables

When the given project will be successfully accomplished, it is assumed that this system will be an intelligent agricultural decision-support system which will not only identify the diseases affecting crops but also provide farmers with effective treatment and prevention methods. The results are not limited to the forecasts they make in the long-term advancement of digital farming, sustainable land use, and elimination of reliance on manual diagnosis and outside specialists. The system with the incorporation of AI is likely to be able to provide quick and exceptionally reliable disease identification over the usual visual detection, allowing farmers to respond to the disease before it gets to destroy their product.

Among the significant anticipated outcomes, it is possible to mention the possibility to decrease the duration between the outbreak of the disease and the enactment of the treatment. Rather than having to wait until experts analyze the data or open documentation to achieve this, the user will automatically obtain identification of the disease, as well as steps to take to treat the disease within seconds. This aspect will greatly reduce the percentage of losses due to crop and keep their quality intact, besides ensuring their high productivity in the fields. Because the system will archive prediction history, farmers will be in a position to monitor the trends of the diseases developing with seasons, learning trends and planning preventative measures against the crop growth in the next crop cycles.

I. Expected Functional Outcomes

- The system should accurately identify multiple type of crop diseases through image input.
- Disease result will include:
  i. Name of disease detected
  ii. Confidence level
  iii. Brief description of symptoms
  iv. Treatment steps and fertilizer suggestions
- Farmers will be able to upload images through a simple interface without technical expertise.
- A dashboard will visually highlight common diseases and frequency trends using charts.
- The system will generate real time results within 5 to 7 seconds under normal server load.

II. Expected System Performance Outcomes

- AI model should achieve high accuracy detection rates after training.
- The system should perform under varying network conditions through PWA caching.
- MongoDb must store prediction history reliably without data loss.
- The platform should support hundreds to thousands of user requests simultaneously when hosted on cloud environments.
- Scalability will allow the system to incorporate additional crops, regions and new diseases in the future.

III. Expected Deliverables of the Fyp

Table 2: Table of Deliverable Time and its Description

S.N	Deliverable Type	Description
i.	AI Model	Trained using PlantVillage dataset for disease detection.
ii.	Flask API Service	Handles prediction requests and returns classification results.
iii.	MERN Web Application	Interface for image upload, result display and analytics.
iv.	Admin Dashboard	Controls dataset upload, user management and model updates.
v.	History Tracking Module	Stores all previous predictions with date, time and image.
vi.	Treatment Recommendation Database	Contains fertilizer and organic cure suggestions.
vii.	Full Report and Documention	SRS, ERD, Architecture, Testing report and research analysis.

IV. Broader Advantages of This Outcome

- Reduced crop loss → Improved yield → Better farmer income stability
- Digital awareness and technological adaptation in agricultural communities
- Support for smart farming integration and sustainable environmental practices
- Future expansion into mobile apps and nationwide crop monitoring

In the long-term, the desired result is to change the historical process of disease identification through a guesses solution and convert it into the process of data-driven agricultural decision making which is a practical approach to every farmer based on hearsay, quality of harvest and the sustainability of food security in the long term.

4. Methodology

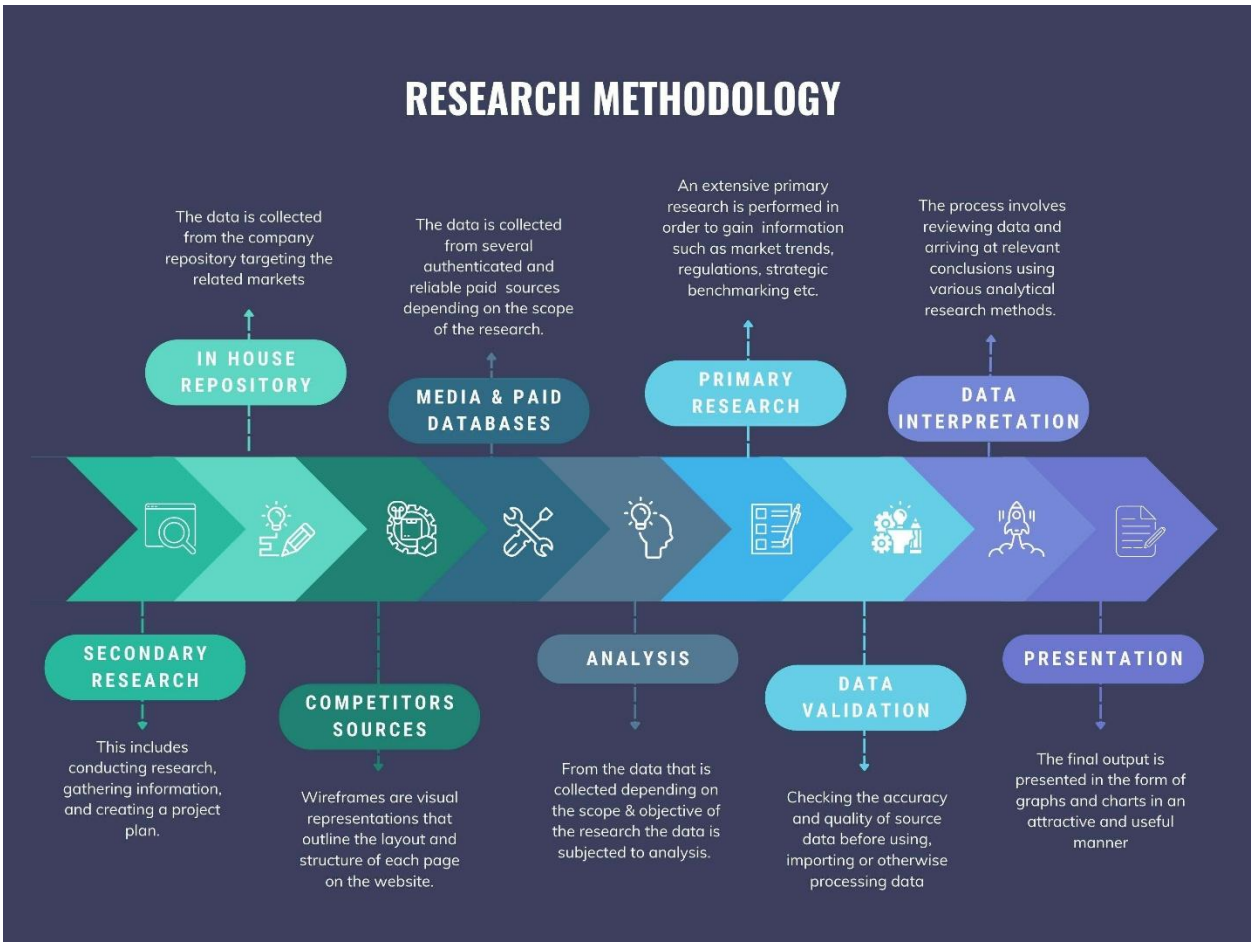


Figure 2: Methodology

Methodology is a system or a procedural model that is employed to design, develop and maintain a software system. It gives a systematic advice on how to plan work, allocate finances, manage time and achieve quality delivery. A methodology explains the procedures, tools, and techniques to be observed during the software development lifecycle, including the requirement gathering stage, to the deployment and maintenance.

In the case of the AI-based Crop Disease Detection System, the methodology is particularly of great importance. The creation of a system that incorporates the concepts of artificial intelligence, computer vision, and software engineering will need proper planning, procedural implementation, and cyclical testing. An appropriate methodology allows the project to be efficiently, organised and predictably developed and risks are minimised and the resources are used well. It is also being able to observe the progress continuously, as well as retain its consistency, to incorporate feedback and believe me this is essential to being able to see that the AI models are correct and the application is easily usable.

Importance of Methodology in FYP

Figure 3: Important of Methodology

The methodology is important as it directs the project towards success. In the case of this FYP, it offers a systematic scheme of planning, designing, developing, and testing the AI-based system. It guarantees effective time management and adequate resource allocation which is necessary to handle various elements including data collection, AI model training, backend development and front end design. With the help of a strict methodology, possible risks will be estimated and eliminated in the first phase before a delay or mistake is committed.

Also, the methodology encourages the progressive development and the system can be tested and improved on the basis of the feedback about the advisors, the potential users, or the other stakeholders. This is especially essential in the case of the AI component where the model performance and accuracy of prediction should be evaluated continuously. Scalability and maintainability of the system is also assured with the methodology which enhances systematic documentation, appropriate coding practices and formal development processes.

In summary, adopting an appropriate methodology for the AI-based Crop Disease Detection System ensures that the project progresses in a well organized, efficient and flexible manner**,** while delivering a high-quality, reliable and practical solution for farmers. It enables the integration of AI models, software components and user interface features in a coordinated way by ensuring that the final system meets both technical and functional requirements of the FYP.

4.1. Considered Methodologies

Before selecting a methodology several approaches were evaluated for their suitability:

- Waterfall Model

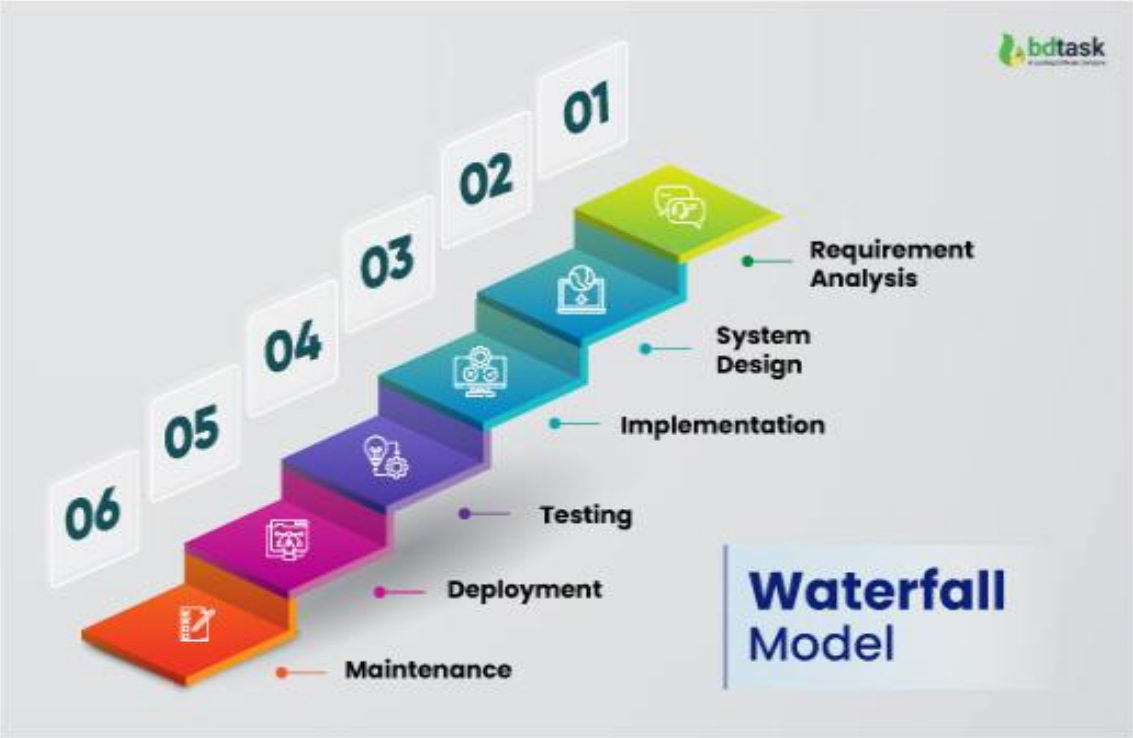


Figure 4: Waterfall Model

The Waterfall model is a traditional software development methodology that follows a linear and sequential process. Each phase of development—requirement analysis, design, implementation, testing, deployment, and maintenance—must be completed before moving to the next. The primary advantage of this model is that it provides a clear roadmap for project development, making it easier to manage, document, and track progress. It is highly structured and often suitable for projects with well-defined requirements that are unlikely to change.

However, for a project like ours, which involves AI model training and iterative testing, the Waterfall model has significant limitations. Any change in requirements during later stages would require revisiting earlier phases, leading to time delays and increased costs. Furthermore, the lack of intermediate deliverables makes it difficult to assess progress or gather user feedback until the final product is developed. While Waterfall is simple to understand and implement, its inflexibility makes it less suitable for projects requiring experimentation, frequent testing, and adaptation, such as an AI-based detection system.

- Agile Methodology

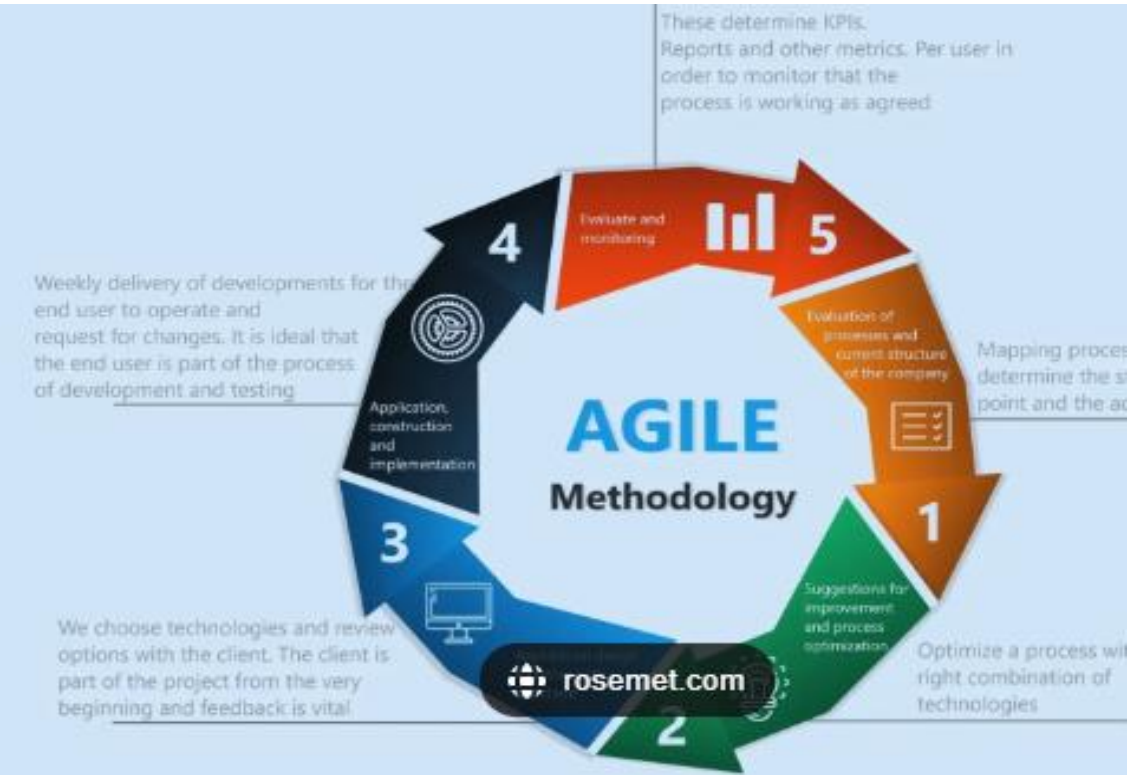


Figure 5: Agile Methodology

The Agile methodology emphasizes iterative development, collaboration, and flexibility. Unlike Waterfall, Agile does not follow a strict sequential approach; instead, it breaks the project into small, manageable increments called iterations. Each iteration produces a working component, which is reviewed and tested before proceeding to the next stage.

For our AI-based Crop Disease Detection System, Agile provides several advantages. First, it accommodates changing requirements, which is essential because AI model performance depends on the dataset quality, testing feedback, and model optimization. Second, Agile promotes collaboration and communication among developers, which ensures that all aspects, such as frontend, backend, and AI integration, are synchronized. Third, it allows for incremental delivery, so components like the dataset preparation, model training, and application interface can be developed, tested, and improved iteratively.

The primary challenge of Agile is that it requires continuous stakeholder engagement and disciplined team management. Without regular feedback, there is a risk of diverging from project goals. Nevertheless, Agile's iterative nature makes it highly suitable for projects involving experimental AI models and evolving software requirements.

- Scrum Framework

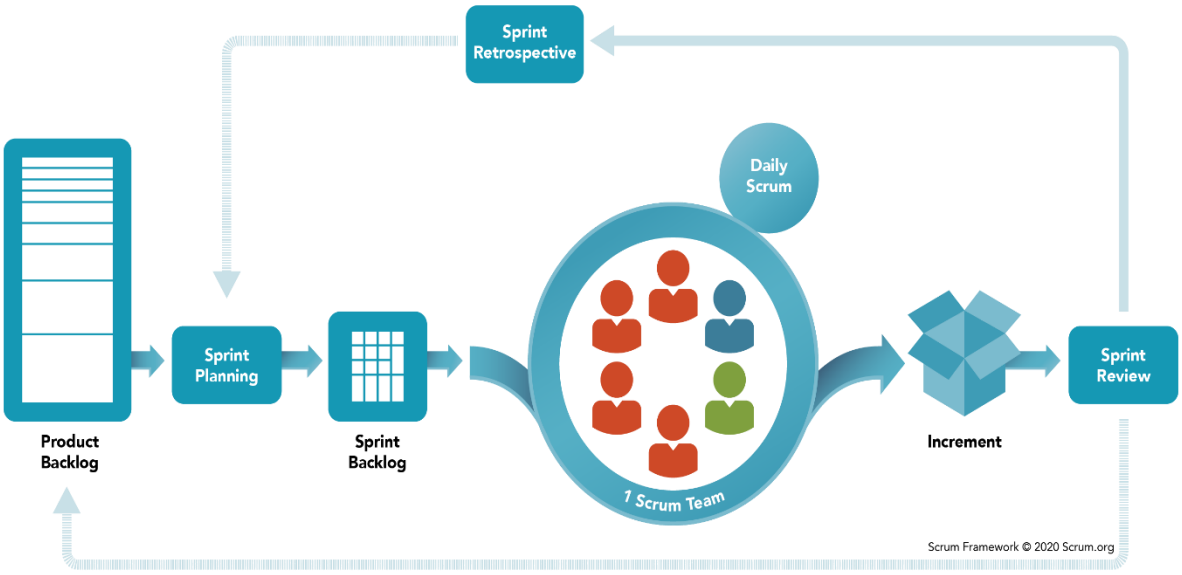


Figure 6: Scrum Framework

Scrum is a specific implementation of Agile, focusing on short development cycles called sprints, typically lasting 1–4 weeks. Each sprint is aimed at delivering specific features or functional components, followed by a review and retrospective session. Scrum emphasizes team collaboration, transparency, and regular feedback, ensuring the project remains aligned with objectives.

In our project, Scrum allows us to plan sprints around AI model development, frontend and backend integration, testing, and deployment. Daily standups facilitate communication, helping to resolve obstacles immediately. Sprint reviews and retrospectives allow continuous improvement, ensuring the final system meets both technical requirements and user expectations. While Scrum requires active participation and discipline, its focus on iterative development, collaboration, and risk reduction makes it ideal for an AI-based software project where uncertainties are common.

- Rapid Application Development (RAD)

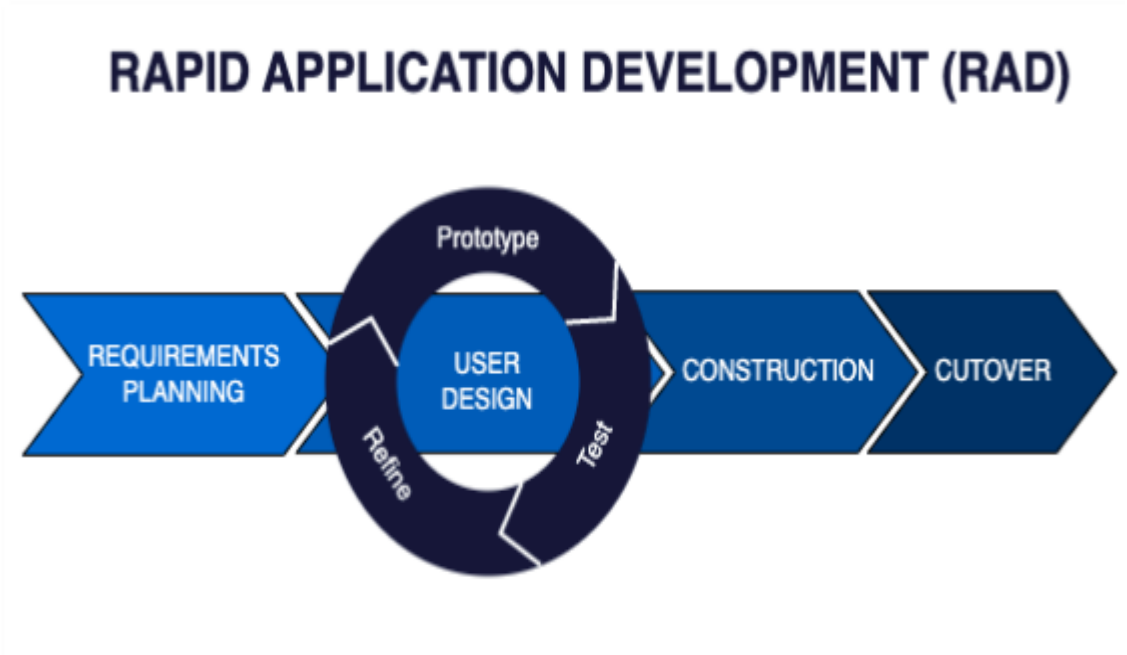


Figure 7: Rapid Application Development

RAD prioritizes quick prototyping and iterative feedback to accelerate project delivery. Unlike traditional methodologies, RAD emphasizes building prototypes early and refining them based on user feedback. For software projects, RAD allows developers to experiment with AI algorithms, interface designs, and integration methods before finalizing the solution.

For our system, RAD would help validate the AI model and interface design quickly with users or advisors. However, RAD can compromise on quality if iterations are rushed, and it relies heavily on skilled developers to produce functional prototypes rapidly. While RAD encourages experimentation and speed, it may not provide the structured planning necessary for large-scale AI integration.

- Prototyping Model

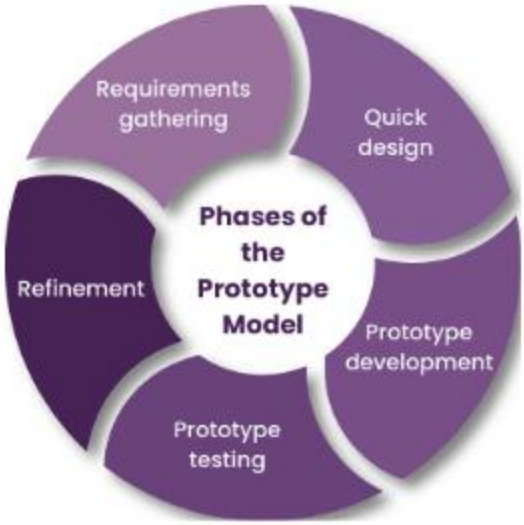


Figure 8: Prototyping Model

The Prototyping model focuses on building a preliminary version of the system to gather user requirements and feedback early. The prototype may not contain full functionality but demonstrates core features, such as AI-based detection and interface usability.

This model is beneficial for understanding user needs, refining system requirements, and validating AI model predictions. However, excessive time spent on prototypes can delay actual development. Additionally, prototypes may create misleading expectations if users assume it represents the final product. Despite these challenges, prototyping is valuable in research-oriented or innovative projects like ours, where user feedback and experimentation play a critical role.

4.2. Selected Methodology

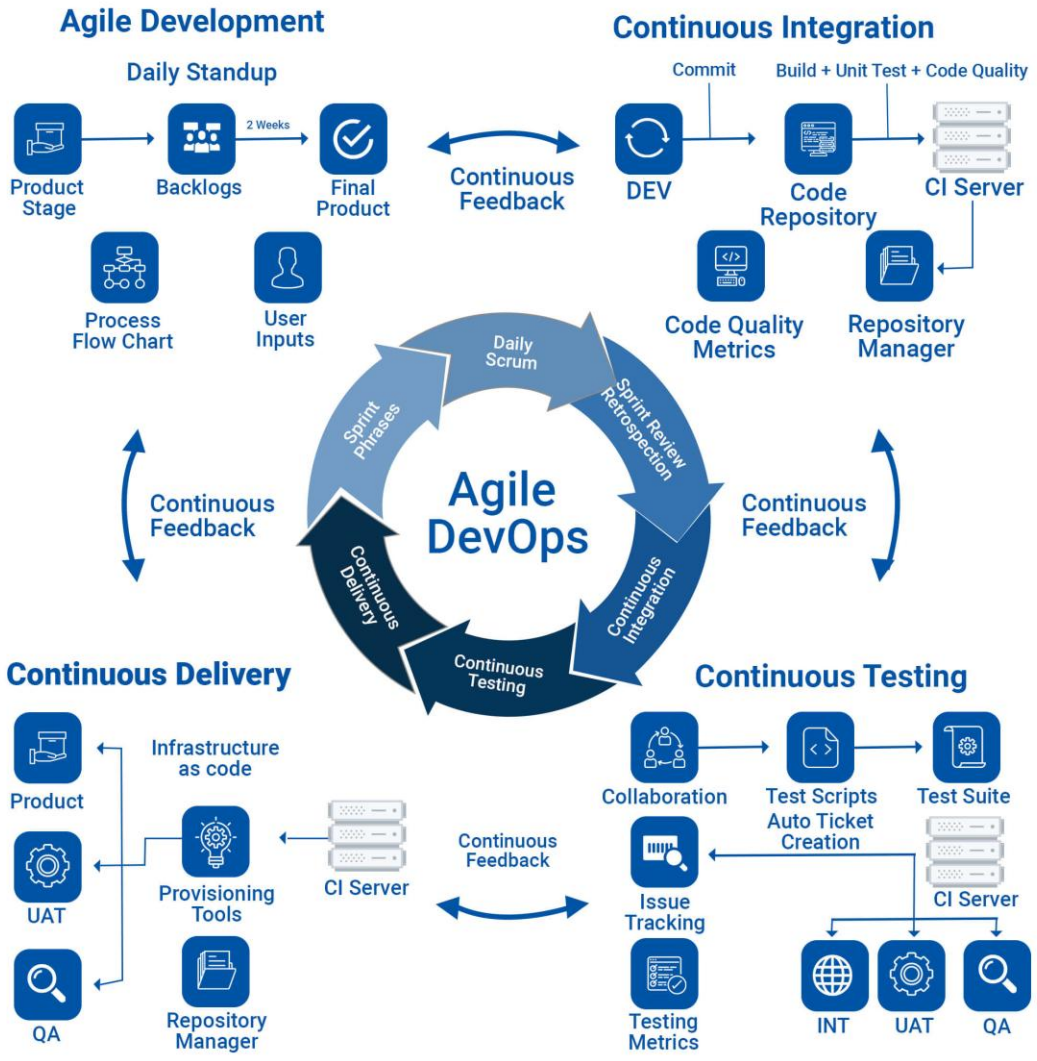


Figure 9: Selected Methodology

After carefully analyzing all the considered methodologies, the Scrum Framework under the Agile methodology was selected as the most suitable approach for the AI-based Crop Disease Detection System. This choice was driven by the project's iterative nature, the need for adaptability, and the structured management approach that Scrum provides. The development of this system involves multiple complex components, including AI model experimentation, software integration, and user interface development. Each of these components requires continuous testing, validation, and feedback, which Scrum accommodates exceptionally well through its sprint-based iterative cycles. Unlike traditional linear methodologies, Scrum allows flexibility to adjust

requirements and refine deliverables based on ongoing evaluation and stakeholder input, which is essential when working with AI models that may need repeated tuning for optimal accuracy.

Scrum organizes the project into short, time-boxed iterations called sprints, each lasting a few weeks and focusing on delivering specific functionalities. For this project, sprints are planned around critical milestones such as dataset preparation, AI model training, frontend and backend development, system integration, and testing. Breaking the work into manageable increments ensures that progress is measurable and any potential issues are identified early, preventing delays or major setbacks. Daily standup meetings provide a platform for immediate communication among team members, facilitating quick resolution of obstacles and enhancing collaboration. At the end of each sprint, sprint reviews and retrospectives are conducted to evaluate completed work, gather feedback, and plan improvements for the next sprint. This continuous review process allows for refinement of both the AI models and the application interface based on performance results and user feedback, ensuring that the system evolves toward optimal functionality.

The adoption of Scrum provides several key benefits for this project. Firstly, flexibility allows the team to adapt to evolving requirements, unforeseen challenges, and iterative improvements in AI model accuracy. Secondly, incremental delivery ensures that functional components are available for testing and evaluation at the end of each sprint, providing tangible results and early validation. Thirdly, Scrum encourages collaboration and communication among all stakeholders, which is critical for aligning AI performance, software development, and user interface design. Additionally, Scrum supports risk management by identifying and addressing issues early, thereby reducing the likelihood of major failures during the project. Finally, the methodology facilitates user feedback integration, allowing potential users, advisors, or stakeholders to test early versions of the system, ensuring that the final product meets real-world needs effectively.

By applying the Scrum Framework, the development of the AI-based Crop Disease Detection System is structured, efficient, and adaptable. It ensures that the project progresses in a coordinated and organized manner while maintaining high quality and timely delivery. The iterative approach enables continuous improvement, better integration of AI and software components, and a user-centered design, making Scrum the most appropriate methodology for this innovative and technically challenging project.

5. Resource Requirements

The AI-Based Crop Disease Detection and Recommendation System implies the presence of structured set of resources such as hardware, software schemes, database tools, cloud services and other utilities. This is guaranteed by the fact that these resources facilitate unhindered development, high-performance model training and scalable deployment. Because the project is concerned with image processing, big data and computationally expensive CNN-based training, resource planning is necessary in order to get big accuracy, responsiveness and long-term maintainability.

Some of the stages that the project will undergo will include development, training, testing, deployment and real world expansion. The stages require various computing power and software support. The resources chosen below assure that the system will operate well not only at the research stage but can also scaled up to be a fully functional agricultural support system to the farmers.

5.1. Hardware Requirements

The hardware resources are split into the development hardware, and field data collection devices and deployment infrastructure. In the development stage, coding and dataset preprocessing as well as deep learning experiments are supposed to be done in a workstation that has adequate processing power and memory. At least, Intel i5/Ryzen 5 processor and 8GB RAM will be sufficient, though 16 up to 32GB RAM and i7/Ryzen 7 processor are also suggested as needed in case of heavy multitasking. SSD storage (minimum 256GB) is required to store images, checkpoints and logs whereas 512GB+ storage is optimal when expanding the dataset. The inclusion of GPU acceleration like NVIDIA GTX1650 or RTX series is optional though very advantageous as CNN training time is greatly reduced by a GPU as compared to CPU.

In case of expansion of real world data, mobile cameras and DSLR cameras may be employed to take pictures of plant leaf. These are tools that maximize variety and strength of the model dataset. When deploying, there will be need of cloud servers with 2 vCPU and 4-8GB RAM with 20-50GB of storage capacity. The server instances that are enabled with GPU can be utilized in case of a faster prediction in case the user base grows. Cloud hosting makes the system globally accessible and supports scalability, CDN caching and automated backups.

Table 3: Table of Hardware Requirement

S.N.	Stage	Hardware	Minimum Requirement	Recommended	Purpose
1.	Development Workstation	CPU, RAM, Storage	i5/Ryzen 5, 8GB RAM, 256GB SSD	i7/Ryzen 7, 16–32GB RAM, 512GB+ SSD	Model training, dataset processing, coding.
2.	Training Acceleration	GPU	GTX 1050/1650	RTX 2060/3060 +	5–20x faster training cycles.

3.	Field Data Collection	Camera Equipment	Smartphone 12MP +	DSLR + Tripod	Real-world dataset expansion.
4.	Cloud Hosting	Server Sepecifications	2 Vcpu, 4GB RAM	4-8GB RAM with GPU option	Scalable deployment & public access

5.2. Software Requirement

The system depends on software resources as a main engine. The machine learning interface will be developed in Python 3.10+ and deep learning models in the form of CNN models that will be trained on TensorFlow/Keras and image segmentation and preprocessing on OpenCV. The matrix operations and the clean up of the dataset will be handled by using NumPy and Pandas whereas Matplotlib/Seaborn will assist in visualizing the training accuracy, loss curves and model improvements.

The backend server will be written in Node.js using Express.js to route and authenticate as well as Flask or FastAPI to provide the trained model in the form of a microservice. This separation makes sure inferences are fast, and there is high-quality communication between the frontend and the AI core. Postman will be used to test API, and secure user authentication will be offered by JWT.

React.js and TailwindCSS will be used to develop the frontend and style it respectively. To visualize the graphical analytics like disease occurrence trends and accuracy results, Recharts or Chart.js will be deployed. The system will be set up as a Progressive Web Application (PWA) that enables partial offline functionality that can be handy in the farmlands where the internet is unreliable.

MongoDB Atlas will be used as the database layer to store user accounts, prediction history, timestamps and feedback. The image storage can be through gridFS or Cloudinary when the size of the file exceeds the usual capacity of the database. Mongoose ORM will manage query and schema design, which will offer data consistency and stability.

Table 4: Table of Software Requirement

S.N.	Category	Tool/Technologies	Purpose
1.	AI & ML Frameworks	Python, TensorFlow/Keras, OpenCV, NumPy, Pandas, Matplotlib.	Training, preprocessing, visualization.
2.	Backend & APIs	Node.js, Express, Flask/FastAPI, Postman, JWT Auth.	API handling, authentication, model serving.
3.	Frontend UI	React.js, TailwindCSS, Chart.js/Recharts, PWA configs.	Interface design, analytics, offline support
4.	Database & Storage	MongoDB, Mongoose, GridFS/Cloudinary.	Peristent storage + image management.

5.3. Additional Utilities

There are also development utilities which optimize effectiveness and quality of projects. The VS Code will be the primary IDE, and it will be supported by the Python and JavaScript and debugging plugins. Version control, collaboration and deployment pipelines will be handled by Git + GitHub. It can be developed in later stages to containerize backend and ML inference models to allow easy deployment. It can be trained on Google Colab or Kaggle GPU that can be free of charge.

Table 5: Table of Additional Utilities

S.N.	Utility	Use
1.	VS Code	Development environment
2.	Git	Version control and collaboration
3.	Google colab / Kaggle	Free GPU training support

All of these resources provide a robust and scalable system of crop disease detector. Training is made faster with the use of a GPU and is accessible worldwide with cloud hosting. The frontend and backend developed on MERN allow quick communication between AI and the user, whereas MongoDB allows the storage of data safely and scalably. Other utilities enhance teamwork, deployment and debugging. Collectively, these resources establish a solid base that can be utilized to conduct research, real-time applications, mass implementation, and future improvements like multi-crop classification, multi-lingual voice assistance and enhanced insights of disease prediction.

6. Work Breakdown Structure (WBS)

The WBS is one of the project management strategies which enable us to divide a big project into small manageable parts. The WBS in our AI based Crop Disease Detection System is like a road map which allows to split the system into stages, tasks and micro tasks, making the planning and responsibility assignment as well as cost estimation and progress tracking more effective. Rather than perceiving the project as a single big workload that the WBS assists us in traversing it step by step with starting with general objectives and then to specific tasks. It takes a hierarchy in which we begin with the big picture at the top with the overall project objective and work our way down to the minor levels of tasks. The first level outlines the project itself that is the AI based Crop Disease Detection System development. It is further subdivided into major deliverables on the second level that include Project Management Requirements Analysis System Design System Development Testing and Validation and Deployment and Maintenance. The third tier breaks down every deliverable into sub tasks such as resource allocation stakeholder analysis UI UX design AI model development integration testing and others. Lastly fourth level, is the smallest work unit that involves the collection of datasets into the cloud system deployment setup interface design field testing bug fixing, and system updates. This is how the whole WBS structure is carried over to be not a one goal but a big deliverables then down to small units of work that are executable and make up a well-defined workflow that will be followed throughout the development process.

Hierarchy Levels in WBS

- Level 1 – Project Title / Goal
    a. AI-based Crop Disease Detection System

- Level 2 – Major Deliverables
    a. Project Management
    b. Requirements Analysis
    c. System Design
    d. System Development
    e. Testing & Validation
    f. Deployment & Maintenance

- Level 3 – Sub-deliverables

    i. Project Management:

        a. Planning & Scheduling

b. Resource Allocation
c. Risk Management

ii. Requirements Analysis:

a.Stakeholder Identification

a. Functional Requirements
b. Non-Functional Requirements
c. Data Collection

iii. System Design:

a. Architecture Design (Layered/MVC)
b. Database Design
c. UI/UX Prototyping

d. AI Model Design

iv. System Development:

a. Frontend Development
b. Backend Development
c. AI Model Training
d. API Integration

v. Testing & Validation:
a. Unit Testing
b. Integration Testing
c. Performance Testing
d. User Acceptance Testing

vi. Deployment & Maintenance:
a. Cloud Deployment
b. User Training
c. Bug Fixing & Updates
d. Future Enhancements

- Level 4 – Work Packages

These are the smallest, actionable tasks that teams will actually perform.

Table 6: Table of WBS Sub deliverable and its description

S.N.	Sub Deliverable	Work Package	Task Description
1.	Planning and Scheduling	Project Timeline Creation	Develop Gantt chart, establish milestones
2.	Resource Allocation	Assignment	AI trainers, testers
3.	Data Collection	Dataset Preparation	Gather healthy and diseased plant images
4.	AI Model Training	CNN Model Training	Select and train suitable deep learning model
5.	UI/UX Prototyping	Figma Designing	Create app/web interface prototypes
6.	Integration Testing	API and UI Testing	Ensure responsive connection between components
7.	User Acceptance Testing	Field Validation	Test with farmers, collect, real use feedback
8.	Deployment	Cloud Server Setup	Configure AWS/Azure hosting setup
9.	Maintenance	Continuous Updates	Fix bugs, optimize model and UI over time

At last what I want to say is that i.e. The Work Breakdown Structure is the backbone of our project execution. It transforms a complex AI-based system into organized units, making development more manageable and structured. By clearly identifying tasks, responsibilities, and workflow, WBS guides us from planning to deployment smoothly and minimizes chances of delay or confusion. It ensures we progress step-by-step until we successfully complete the project.

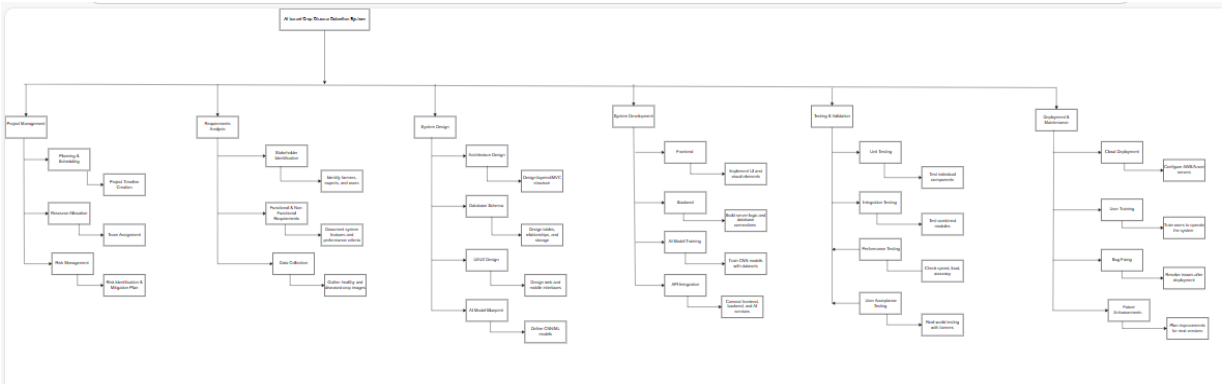


Figure 10: WBS

7. ERD

The ERD for the AI-Powered Crop Disease Detection and Recommendation System presents the structural layout of the database layer supporting all the AI-driven prediction operations, user roles, farming knowledge-base storage, community interaction, and system analytics. Each entity defines a data object in the system along with its properties, and relationship mapping ensures seamless workflow between prediction, treatment suggestion, calendar management, and expert advisory support.
The architecture is scalable and supports various user categories, model upgrades, dataset retraining, automated notifications, and role-based data access.

7.1. Entities and Attributes

In a database system, entities represent the primary objects or components about which data is stored. These can be people, objects, events, places, or concepts that exist within the system and are of some significance to the application. Each entity will be considered as an independent data unit, and in a relational database, it is normally mapped to a table. For instance, in an AI-powered crop disease detection system, major data elements such as User, Crop, Disease, Farmer, and AI_Model are the entities that the system interacts with and maintains information regarding.

Attributes, on the other hand, describe characteristics or properties of each entity. They define what type of information is stored within an entity, and in a database, they would be represented as fields or columns within a table. Each entity contains multiple attributes in order to uniquely define its behavior and structure. Such an example could be the User entity, which might have full_name, email, contact_number, and role as its attributes. These define the particular identity and details of any user. Put simply, entities describe what the system is storing data about, while attributes describe what detail of that entity is being stored.

1. User

- user_id (PK)
- full_name
- email
- password_hash
- contact_number
- role
- date_created

2. Admin

- admin_id (PK)
- user_id (FK)
- permissions
- last_login
- created_at

3. Expert

- expert_id (PK)
- user_id (FK)
- specialization
- certification_id
- verified_status
- rating

4. Farmer

- farmer_id (PK)
- user_id (FK)
- location
- land_size
- preferred_crop

5. Crop

- crop_id (PK)
- crop_name
- type
- growth_duration
- suitable_climate
- soil_type

6. Disease

- disease_id (PK)
- disease_name

- crop_id (FK)
- severity_level
- description
- common_symptoms

7. AI_Model

- model_id (PK)
- model_version
- training_dataset
- accuracy_score
- deployment_status
- updated_at

8. Prediction

- prediction_id (PK)
- user_id (FK)
- disease_id (FK)
- confidence_score
- image_path
- prediction_timestamp

9. Treatment_Recommendation

- recommendation_id (PK)
- disease_id (FK)
- treatment_type
- product_name
- dosage_instruction
- eco_friendly_option

10. Fertilizer

- fertilizer_id (PK)
- fertilizer_name
- crop_id (FK)
- soil_requirement

- application_method
- recommended_season

11. Pesticide

- pesticide_id (PK)
- product_name
- chemical_composition
- application_frequency
- safety_precautions
- disease_id (FK)

12. History_Record

- record_id (PK)
- user_id (FK)
- prediction_id (FK)
- status
- notes
- record_created

13. Crop_Calendar

- calendar_id (PK)
- crop_id (FK)
- planting_month
- harvesting_month
- season_alerts

14. Notification

- notification_id (PK)
- user_id (FK)
- message
- notification_type
- status
- created_at

15. Forum_Post

- post_id (PK)
- user_id (FK)
- title
- content
- image_path
- created_date

16. Forum_Reply

- reply_id (PK)
- post_id (FK)
- user_id (FK)
- reply_content
- created_at
- upvotes

17. Dataset

- dataset_id (PK)
- source_name
- total_images
- labels_count
- last_updated

18. Analytics_Report

- report_id (PK)
- admin_id (FK)
- report_type
- generated_date
- affected_region
- key_metrics

19. Region

- region_id (PK)

- region_name
- climate
- humidity_level
- soil_ph
- risk_index

20. Subscription

- subscription_id (PK)
- farmer_id (FK)
- plan_type
- payment_status
- valid_until

7.2. Relationship Mapping (Well-Formatted + Clear Representation)

- User Role Relationship
  i. A User account can exist as a Farmer, an Expert, or an Admin
  ii. **Type:** Inheritance
  iii. **Cardinality:** 1 User = 1 specific role

- Crop & Disease
  i. One Crop may have multiple associated Diseases
  ii. **Type:** One-to-Many
  iii. **Cardinality:** Crop (1) → Disease (Many)

- Disease & Recommendations
  i. One Disease can have several Treatment Recommendations
  ii. **Type:** One-to-Many
  iii. **Cardinality:** Disease (1) → Recommendation (Many)

- Disease & Pesticides
  i. A Disease may require more than one Pesticide product
  ii. **Type:** One-to-Many
  iii. **Cardinality:** Disease (1) → Pesticide (Many)

- User/Farmer & Prediction
  i. A single user (or specifically a farmer) can upload multiple Predictions
  ii. **Type:** One-to-Many
  iii. **Cardinality:** User (1) → Prediction (Many)

- Prediction & Disease
  i. Each Prediction corresponds to one Disease result
  ii. **Type:** Many-to-One
  iii. **Cardinality:** Prediction (Many) → Disease (1)

- Prediction & History Record
  i. Every Prediction has one and only one History Record
  ii. **Type:** One-to-On
  iii. **Cardinality:** Prediction (1) → History Record (1)

- Crop & Crop Calendar
  i. Each Crop maintains one Crop Calendar schedule
  ii. **Type:** One-to-One
  iii. **Cardinality:** Crop (1) → Crop Calendar (1)

- User & Notifications
  i. One User may receive many Notifications over time
  ii. **Type:** One-to-Many
  iii. **Cardinality:** User (1) → Notification (Many)

- User & Forum Posts
  i. A User is allowed to write multiple Forum Posts
  ii. **Type:** One-to-Many
  iii. **Cardinality:** User (1) → Post (Many)

- Post & Replies
  i. A single Post can have several Replies from users
  ii. **Type:** One-to-Many
  iii. **Cardinality:** Post (1) → Reply (Many)

- Dataset & AI Models
  i. One Dataset can be used to train multiple AI_Model versions
  ii. **Type:** One-to-Many
  iii. **Cardinality:** Dataset (1) → Model (Many)

- Admin & Reports
  i. Each Admin can generate multiple Analytics Reports
  ii. **Type:** One-to-Many
  iii. **Cardinality:** Admin (1) → Analytics Report (Many)

- Region & Farmers / Predictions
  i. One Region can be associated with several Farmers and multiple Predictions
  ii. **Type:** One-to-Many
  iii. **Cardinality:** Region (1) → Farmer/Prediction (Many)

- Farmer & Subscription
  i. A Farmer may have one active Subscription plan
  ii. **Type:** One-to-One
  iii. **Cardinality:** Farmer (1) → Subscription (1)

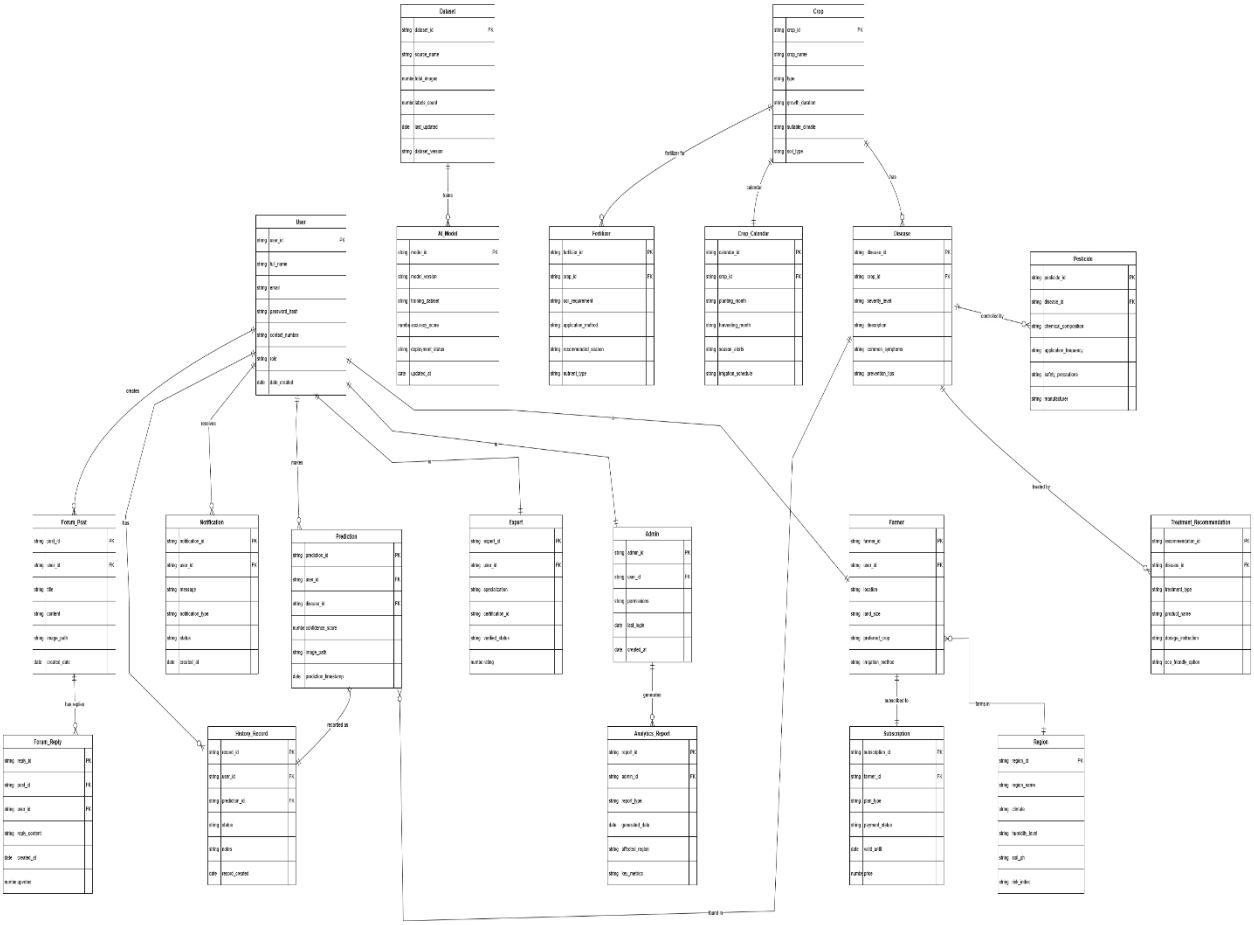


Figure 11: ERD

8. Product Backlog (SRS for Scrum Methodology)

Product Backlog Product Backlog is the list of those features, requirements, additions, or fixes needed in a software system, in order and constantly changing. It is the sole provider of the requirements upon any alterations to be done to the product and is fundamental in the Scrum model. Product Backlog items are usually defined as user stories, a system functionality that is described through the eyes of end users or stakeholders (Schwaber and Sutherland, 2020).

Product Backlog is also prioritized in terms of business value, risk and dependency where the most important features are built at the beginning. Agile development does not have a fixed backlog, but the backlog is enhanced on a regular basis based on the changing user requirements, feedback, and project constraints. When the Sprint Backlog is being planned, items are taken out of the Product Backlog and into the Sprint Backlog, which allows it to be developed incrementally and iteratively.

The functional requirements that are reflected in the Product Backlog in this project include user authentication, AI-based crop disease detection, treatment recommendation, analytics, and administrative controls. The project is also well structured through its Product Backlog, which makes it have transparency, good planning, and adherence to the Agile and Scrum concepts during the development lifecycle.

Table 7: Table of Product Backlog

ID	User Story	Priority	Sprint	Story Point
US001	As a farmer, I want to register an account so that I can use the system securely.	High	1	3
US002	As a farmer, I want to log in using my credentials so that I can access my dashboard.	High	1	2
US0003	As a farmer, I want to upload an image of a crop leaf so that the system can detect diseases.	High	1	5
US004	As a system, I want to preprocess uploaded images so that AI predictions are accurate.	High	1	5
US005	As a system, I want to classify crop diseases using a CNN model.	High	2	8
US006	As a farmer, I want to view disease prediction results so that I know the crop condition.	High	2	3
US007	As a farmer, I want to see a confidence score with predictions so that I can trust results.	Medium	2	2

US008	As a farmer, I want to receive treatment recommendations so that I can cure the disease.	High	2	5
US009	As a farmer, I want organic treatment suggestions so that I can avoid chemical damage.	Medium	3	3
US010	As a farmer, I want pesticide recommendations with dosage so that treatment is safe.	High	3	5
US011	As a system, I want to store prediction history so that farmers can track past results.	High	3	5
US012	As a farmer, I want to view my prediction history so that I can monitor crop health.	Medium	3	3
US013	As a farmer, I want disease trend charts so that I can identify frequent issues.	Medium	4	3
US014	As a system, I want to generate analytics reports so that trends are visualized clearly.	Medium	4	5
US015	As an admin, I want to manage crop and disease datasets so that AI accuracy improves.	High	4	8
US016	As an admin, I want to retrain AI models so that predictions stay up-to-date.	High	5	8
US017	As an admin, I want to manage users so that system access is controlled.	Medium	5	3
US018	As a farmer, I want notifications for disease alerts so that I act on time.	Medium	5	3
US019	As a system, I want to support mobile devices so that farmers can use phones easily.	Medium	6	5
US020	As a system, I want offline support (PWA) so that farmers can use it in low network areas.	Medium	6	5
US021	As a farmer, I want crop calendar suggestions so that I plan farming activities.	Low	6	3
US022	As a farmer, I want to participate in a discussion forum so that I can share knowledge.	Low	7	3
US023	As an expert, I want to reply to farmer queries so that proper guidance is given.	Medium	7	3

US024	As a system, I want secure authentication so that user data is protected.	High	7	5
US025	As a system, I want cloud deployment so that the application is scalable and available.	High	8	8

8.2. Sprint Backlog

The Sprint Backlog is an extension of the Product Backlog that includes the chosen user stories, tasks and technical activities that the development team promises to accomplish within a particular sprint. It is the work to be implemented in a specific duration of time, normally one to four weeks, and developed at the sprint planning meeting.

Sprint Backlog is a short term implementation plan in Scrum methodology. It contains the high-priority user stories selected among the Product Backlog and their approximate effort (story points) and implementation information. Sprint backlog belongs to the development team and is regularly updated as the work is developed during the sprint.

In this project, Sprint Backlog is used to group user stories, including image upload, AI-based disease detection, treatment recommendation, analytics, and system deployment into several sprints. This method guarantees the gradual improvement, constant testing, and initial delivery of fundamental capabilities. With the help of a Sprint Backlog, the project is transparent and better time-managed, and it can constantly adjust itself on the feedback and performance assessment (Schwaber and Sutherland, 2020).

The Major Characteristics of a Sprint Backlog.

- Includes some user stories chosen in the Product Backlog.
- Time based on a definite sprint period.
- Effort estimation uses story points.
- Concentrates on performance and action.
- Change on a daily basis during the sprint.

Sprint 1 – User Access & Core Upload (Foundation Sprint)

Sprint Goal: Enable user registration, login, and image upload.

ID	User Story	Story Point
US001	Farmer registration	3
US002	Farmer login	2
US003	Upload crop leaf image	5

US004	Image preprocessing	5
US024	Secure authentication	5

In Sprint 1 we were working on laying down the main backbone of the system. The primary goal of this sprint was to allow access and basic system interaction without conflicts. In this sprint, we introduced user registration and login system and security authentication systems. We also designed crop image upload option where farmers can post the image of the crop leaves to be analyzed. Further, the process of image preprocessing was also introduced to achieve the desired quality of uploaded images to guarantee the correct disease detection through AI. Every next system functionality was preconditioned by this sprint.

Sprint 2 – AI Disease Detection

Sprint Goal: Implement core AI prediction functionality.

ID	User Story	Story Point
US005	CNN-based disease classification.	8
US006	View prediction results	3
US007	View confidence score	2
US008	Treatment recommendations	5

Sprint 2 was supposed to bring in the essence of intelligence of the system. This sprint involved the incorporation of a Convolutional Neural Network (CNN) model to detect crop diseases using posted images. Other features we created are the ability of the farmers to see disease prediction outcomes and the confidence scores. Moreover, the treatment recommendations were worked out on the detected diseases. This sprint turned the system into a project that is an intelligent decision-support tool instead of being a simple application.

Sprint 3 – History & Recommendation Enhancement

Sprint Goal: Improve usability and monitoring.

ID	User Story	Story Point
US009	Organic treatment suggestions	3
US010	Pesticide & dosage guidance	5
US011	Store prediction history	5
US012	View prediction history	3

We improved the quality and utility of recommendations given to farmers in Sprint 3. The system was spread to provide organic and chemical treatment recommendations including the appropriate dosage. We also added an ability to save the history of predictions and provide the user with an

opportunity to see their previous disease detection outcomes. This sprint enhanced the traceability and aided farmers in tracking crop health with time.

Sprint 4 – Analytics & Admin Control

Sprint Goal: Enable analytics and administrative control.

ID	User Story	Story Point
US013	Disease trend charts	3
US014	Analytics report generation	5
US015	Dataset management	8

Sprint 4 was concerned with data analysis and system management. In this sprint, we installed analytics dashboards, which show disease trends and visual reports. Functionalities like the administration of datasets were also built and enabled administrators to maintain and enhance quality training data. This sprint helped monitor, control, and make data-driven decisions in the system.

Sprint 5 – Model Optimization & Notifications

Sprint Goal: Improve AI accuracy and user engagement.

ID	User Story	Story Point
US016	AI model retraining	8
US017	Admin user management	3
US018	Disease alert notifications	3

Sprint 5 was devoted to enhancing the accuracy of the system and the interaction with the user. The retraining functionality of AI models was introduced to make sure that the disease detection model will be accurate with the arrival of new data. The system access controls were also added by administering the users of the administration. Also, notification of the farmers concerning the important cases of diseases was incorporated to ensure prompt intervention.

Sprint 6 – Mobile & Offline Support

Sprint Goal: Accessibility in rural environments.

ID	User Story	Story Point
US019	Mobile responsiveness	5
US020	PWA offline support	5
US021	Crop calendar suggestions	3

Sprint 6 was aimed at ensuring the system was available in rural and under-connectivity areas. In this sprint, we made the user interface friendly to the mobile device and added Progressive Web App (PWA) features that would allow the user to use it without an internet connection. Crop calendar recommendations were also brought to assist farmers take care of farming activities. This sprint this enhanced the ease of use and access.

Sprint 7 – Community & Expert Interaction

Sprint Goal: Knowledge sharing and expert advice.

ID	User Story	Story Point
US022	Farmer discussion forum	3
US023	Expert replies	3

In the 7 th sprint, we were working on knowledge sharing and expert support. The discussion forum was introduced where farmers should post questions and experiences. Farmers were empowered to answer such questions and offer professional opinions. This sprint improved teamwork and created an encouraging agricultural community of the system.

Sprint 8 – Deployment

Sprint Goal: Make system production-ready.

ID	User Story	Story Point
US025	Cloud deployment	8

The last sprint which is Sprint 8 was devoted to the system deployment and readiness to the real-life application. This sprint saw the application moved to a cloud environment in order to guarantee scalability, availability and performance. The system was also tested, configured and optimized to finalize its functionality and non-functionality requirements. This sprint was the final stage of the development lifecycle.

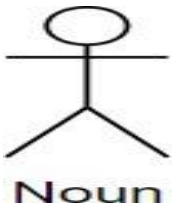
9. UML Diagrams

9.2. Use Case Diagram

A Use Case Diagram is a visual representation of the functional requirements of a system, highlighting the interactions between actors (users or external systems) and the system itself. It captures what tasks the system can perform, who can perform them, and how the system responds to different user actions (Sommerville, 2016). In the AI-Powered Crop Disease Detection and Recommendation System, the use case diagram demonstrates the dynamic interactions between farmers, administrators, agricultural experts, and the AI system. Farmers can submit images of crops to detect diseases, receive automated treatment recommendations, and track the health of their crops over time. Experts can validate disease diagnoses and update treatment suggestions. Administrators manage user accounts, monitor system performance, and ensure data integrity. The AI system acts as a central processing entity, analyzing submitted crop images, detecting diseases using machine learning models, and generating relevant recommendations for treatment. This use case representation helps stakeholders understand the system's functional capabilities, clarifies user roles, and serves as a foundation for system design and development (Jacobson et al., 1992).

9.2.1.Notation used in use case

i. Actor



Actors give the various kinds of users/external systems expected to communicate with the system. They may be individuals, other applications or computer hardware that interact with the system to get things done. The system presents certain roles to different actors and each communicates with the other to reach a goal. Actors in a use case diagram are usually depicted as a stick figure outside the system boundary. As an example, a customer, administrators, and sellers are the actors likely to appear in an e-commerce system. The naming convention is noun.

ii. Use Case

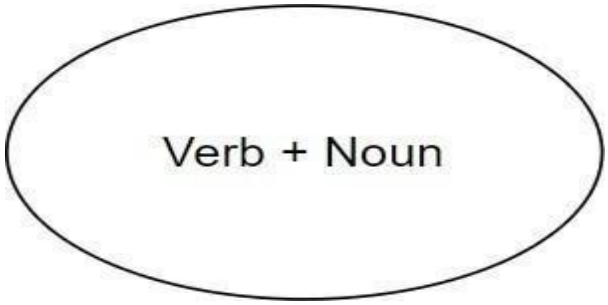


Figure 4: Notation for Use Case

Describing how the system responds to actor requests, use cases detailed the actual actions/tasks performed by the system in response to such requests. Every use case symbolizes a functional requirement, or a specific service that the system provides to the persons. Within the system bound, use cases are shown as horizontal ovals (or ellipses) with the name of the functionality written in them. As an example, use cases could consist of the following in the same context of the e-commerce example: such as Register Account, Place Order, Manage Products, and Generate Sales Report. These assist in explaining what the system should perform on the part of the users.

iii. System boundary

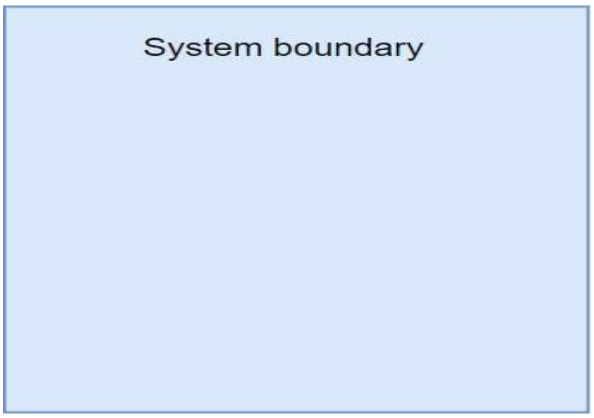


Figure 5: Notation for System boundary

The system boundary controls what is within a system and excludes actors outside a system. It is signified by a great rectangle (box) that contains all the use cases and depicting what belongs to the system or what is outside of it. This is because usually the name of the system is written in the top part of the rectangle. The presented visual representation assists the stakeholders to make a clear distinction between internally

defined behaviors and externally defined behaviors and thus have a well-scoped system design.

iv. Association

Figure 6: Notation for Association

An association is shown by a solid straight line between an actor and a use case. It means that the actor is involved in the use case, either as a producer of the call that leads to use case or as a consumer of some results of the use case. It is in most cases that they do not have an arrowhead as their lines unless the direction is required to be indicated. The use case diagram depends on associations to resemble its foundation based on the direct interaction of the system and the user (or external element).

v. Include

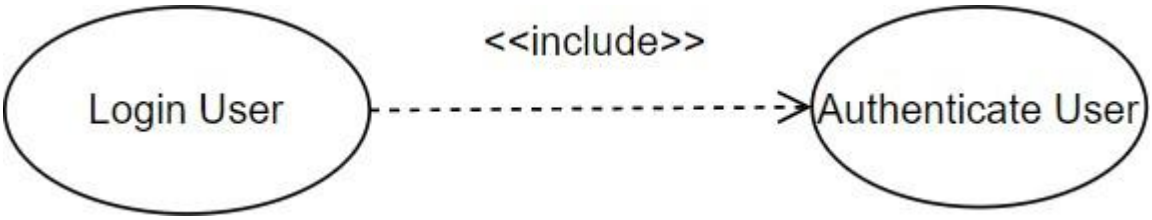


Figure 7: Notation for Include

The include relationship is represented by a dashed arrow labelled «include» and whose direction is drawn as showing the base use case to the included use case. It refers to the fact that the behavior of the included use case will be always executed when executing the base use case. This prevents duplication as it is possible to share functions. In diagrams, the arrow is extended towards the inclusion use case and indicates that such an inclusion is mandatory.

vi. Extends

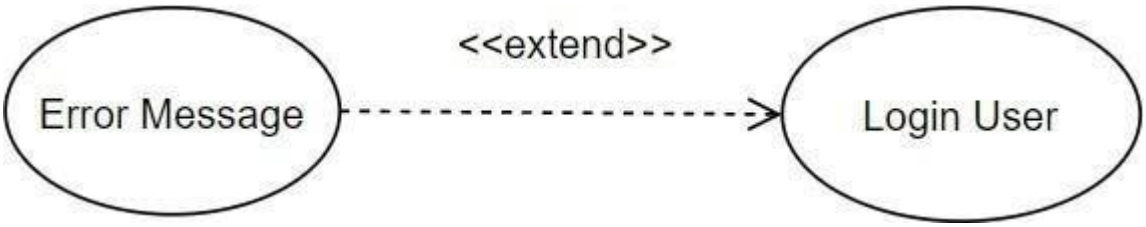


Figure 8: Notation for Extends

The extend association is also illustrated using a dashed arrow but the direction is in opposite as labelling whereby the label is extended and the extended use case points at the base use case. It describes optional or contingent behavior, which can be activated, only under some conditions. It means that the base use case can remain short and extensions can be addressed individually. The direction of the arrow, against which optional functionality, directs to the major covering use case.

vii. Generalization

Figure 9: Notation for Generalization

Generalization is shown as a solid line having a hollow triangle arrow at the end with the more specific actor or use case on the tip. To actors, this implies that all the roles and responsibilities of the parent actor are transferred to the child actor. To be used, it indicates that the specialized use case is the special form of the generalized one. The visual provides reuse of actors/behaviors and hierarchy of actors/behaviors

9.1.2. Actor and Use Case

1. Actors:

- Farmer
- Admin
- Expert
- AI System

1. Use Cases:

   i. Farmer:

- Register
- Login
- Upload Crop Image
- View Disease Detection Result
- View Confidence Score
- View Treatment Recommendations
- View Prediction History
- Receive Disease Notifications
- Participate in Discussion Forum

   ii. Admin

- Manage Users
- Manage Crop Disease Dataset
- Retrain AI Model
- View Analytics Reports

   iii. Expert

- View Farmer Queries
- Reply to Farmer Queries

   iv. AI System

- Preprocess Image
- Detect Crop Disease
- Store Prediction History
- Generate Analytics
- Send Notifications

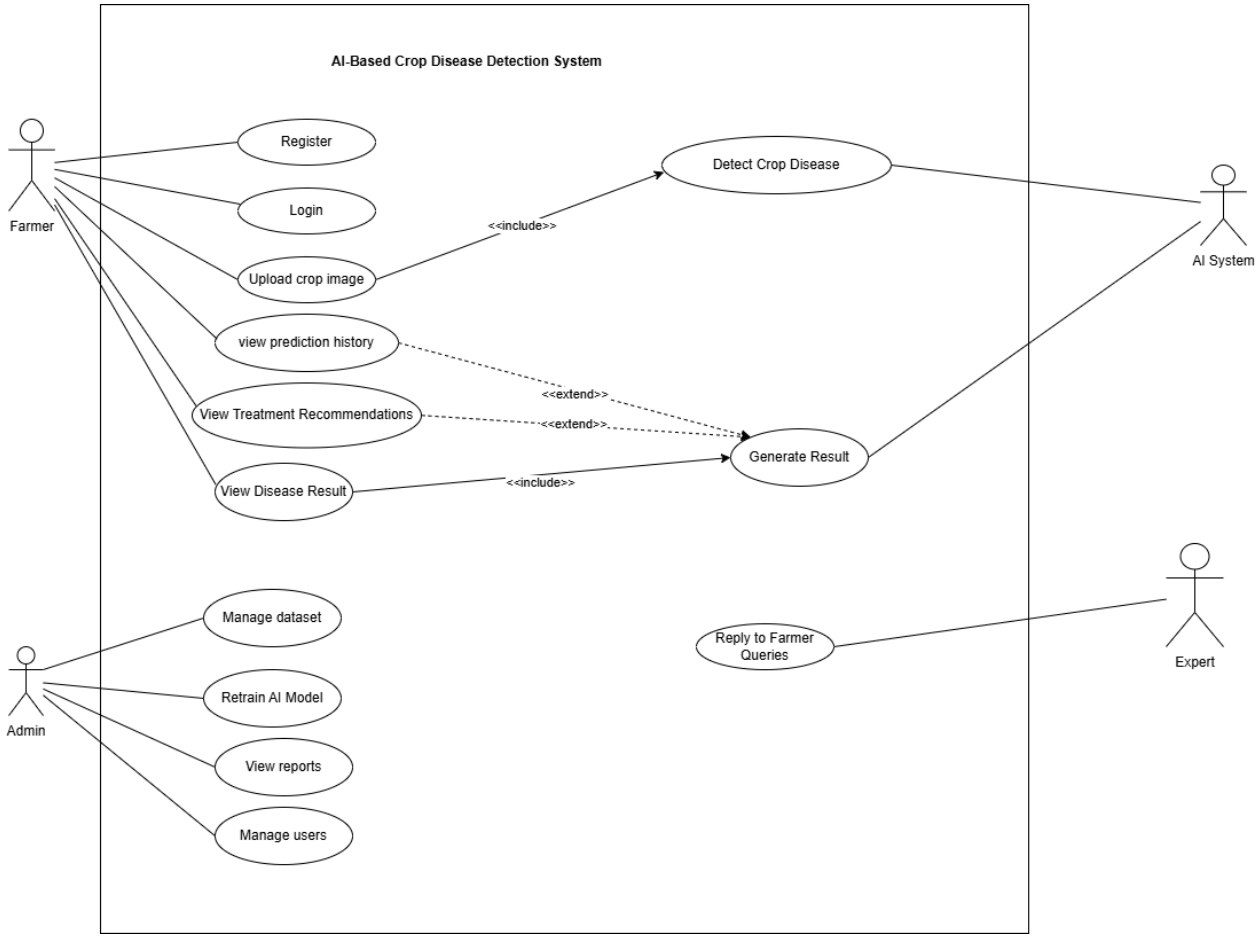


Figure 12: Usecase diagram

Aligned User Stories:
US001, US002, US003, US006, US007, US008, US011, US012, US015, US016, US017, US022, US023, US024

This use case diagram illustrates the interaction between farmers, administrators, experts, and the AI system. It highlights the functional requirements of the system, including disease detection, treatment recommendation, dataset management, and expert interaction.

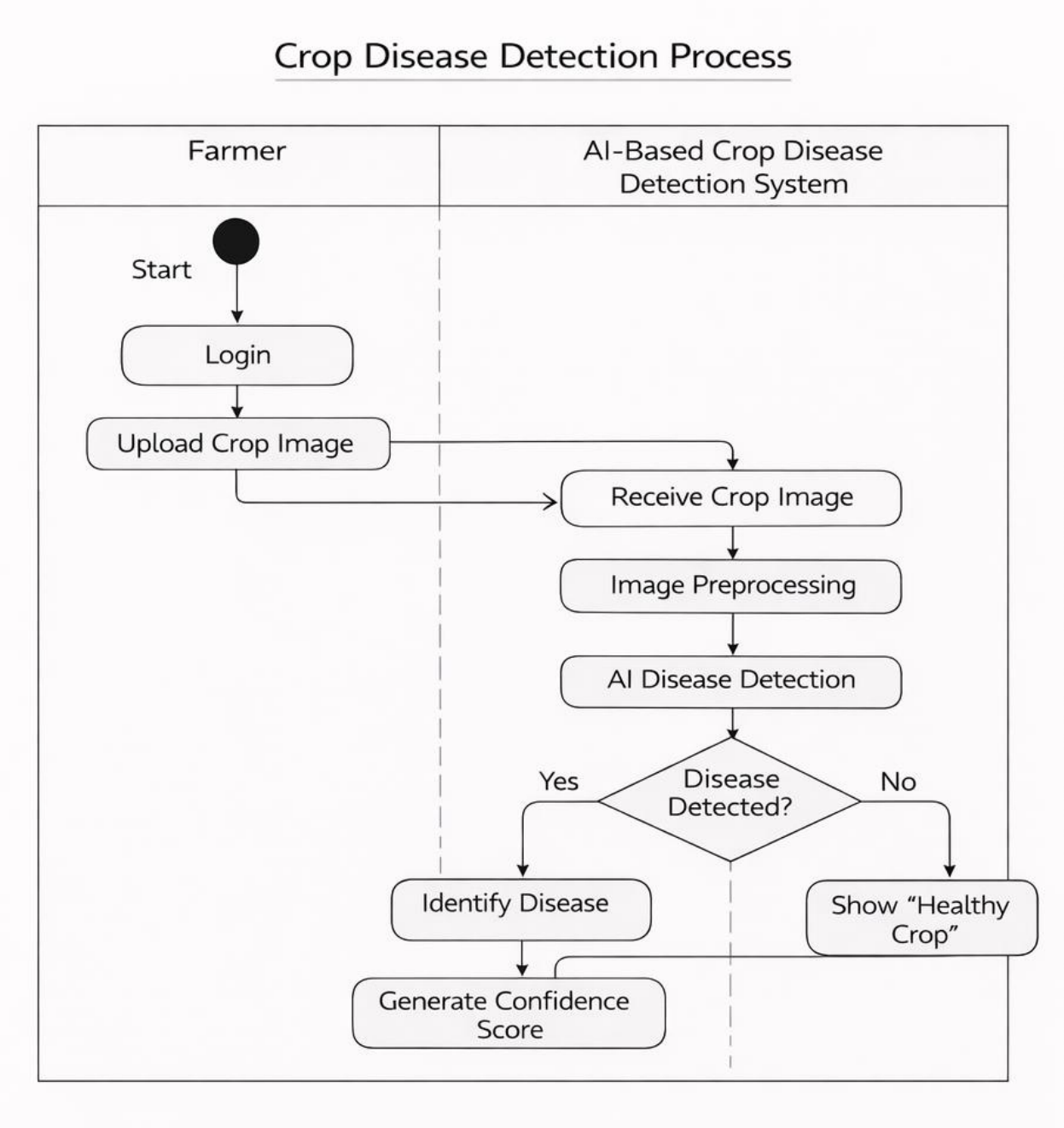
9.2. Activity Diagram

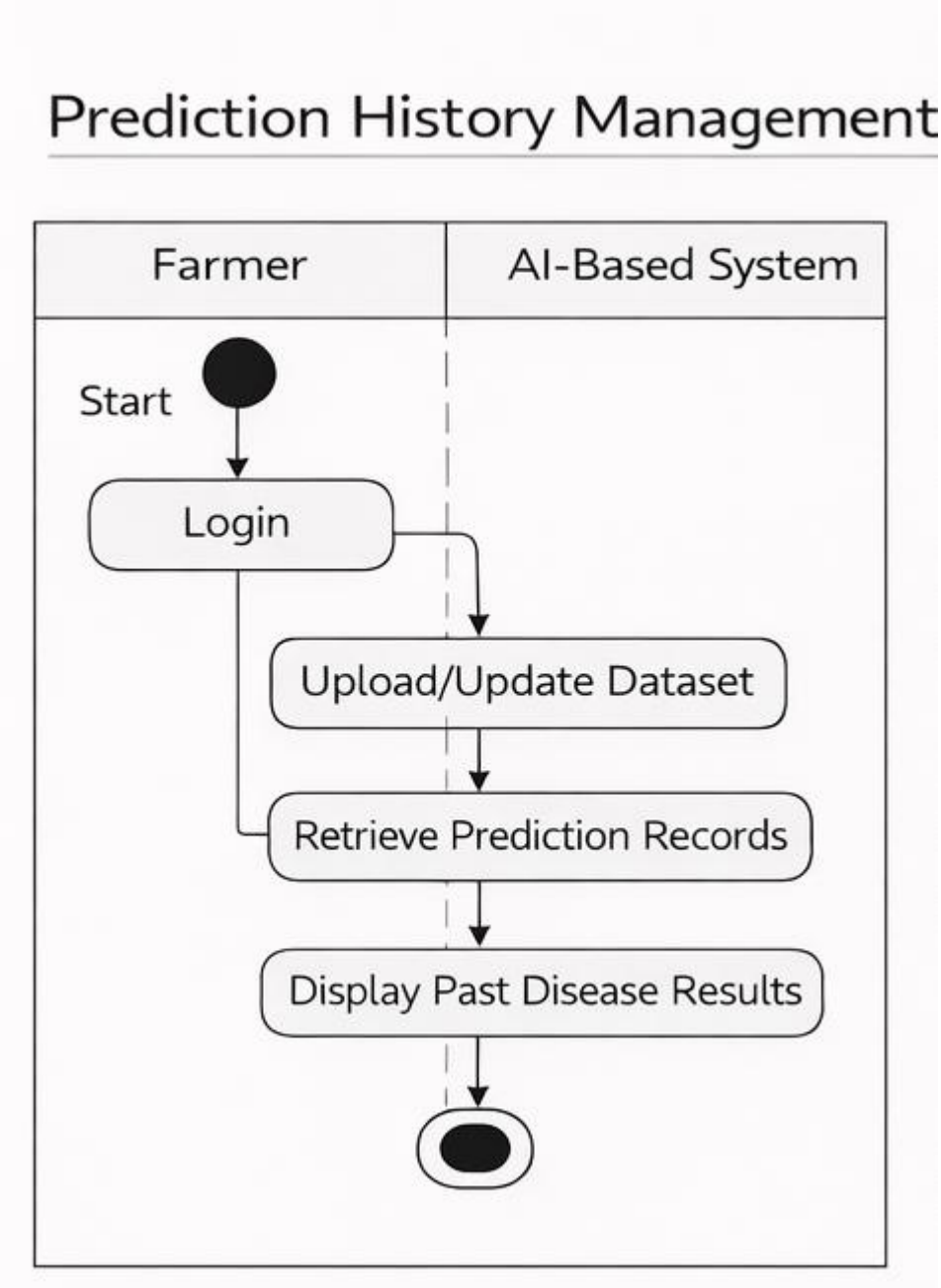
An Activity Diagram is a behavioral diagram in UML that represents the workflow of activities within a system. It shows the step-by-step progression of control from the initiation of a process to its completion, illustrating the sequence of actions, decision points, parallel activities, and synchronization between different tasks (Booch et al., 2005). Activity diagrams are particularly effective for modeling business processes, system workflows, and use case realizations, as they visually clarify how various activities are executed in response to specific events. They provide stakeholders with a clear understanding of the dynamic behavior of the system by illustrating how actions are coordinated and how control transitions between them.

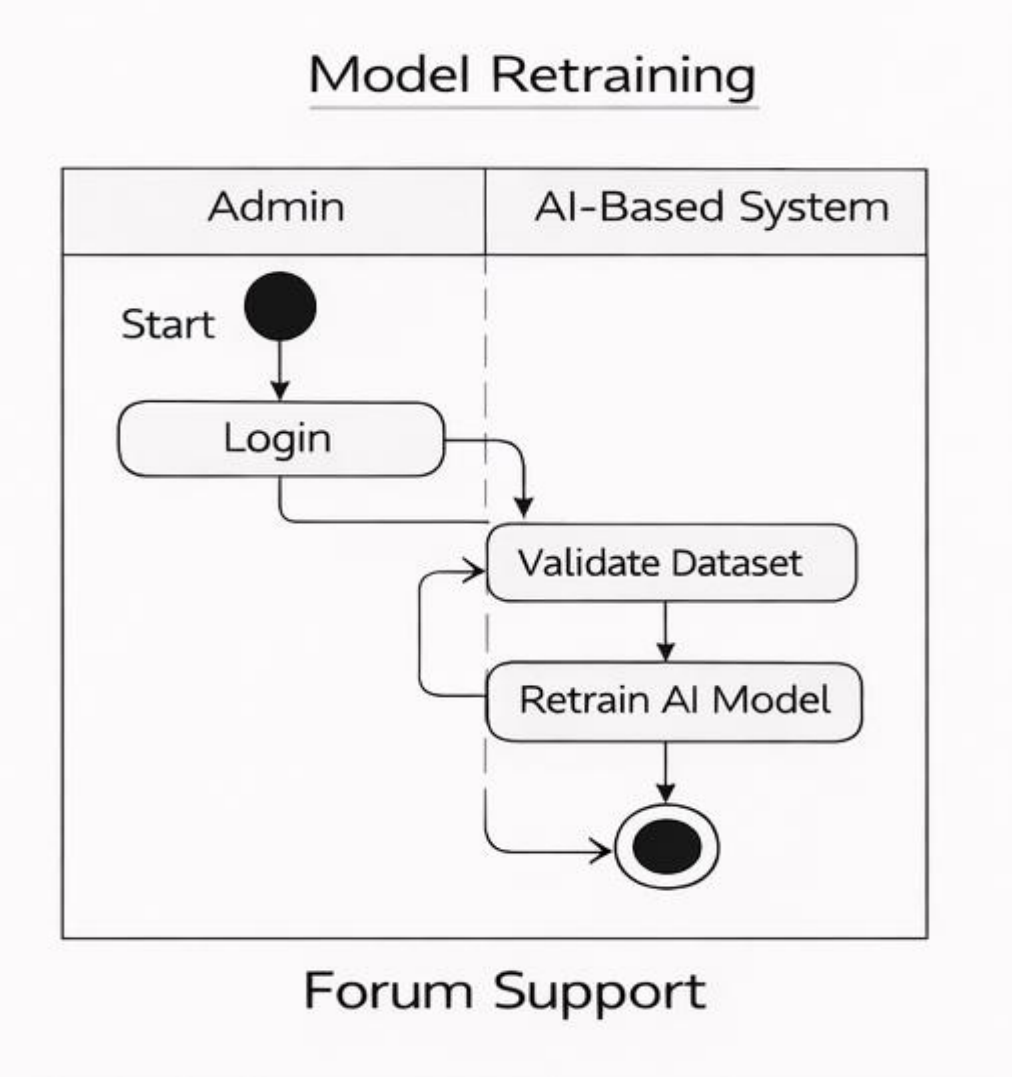
In the AI-Based Crop Disease Detection and Recommendation System, an activity diagram models the workflow beginning with image upload by the farmer, followed **by** image preprocessing**,** disease detection using the AI model**,** result generation, and finally the display of treatment recommendations. By representing these steps, the activity diagram helps ensure a clear understanding of system operations, supports effective system design, and enables validation of the processes, thereby enhancing both usability and reliability of the system (Ambler, 2004).

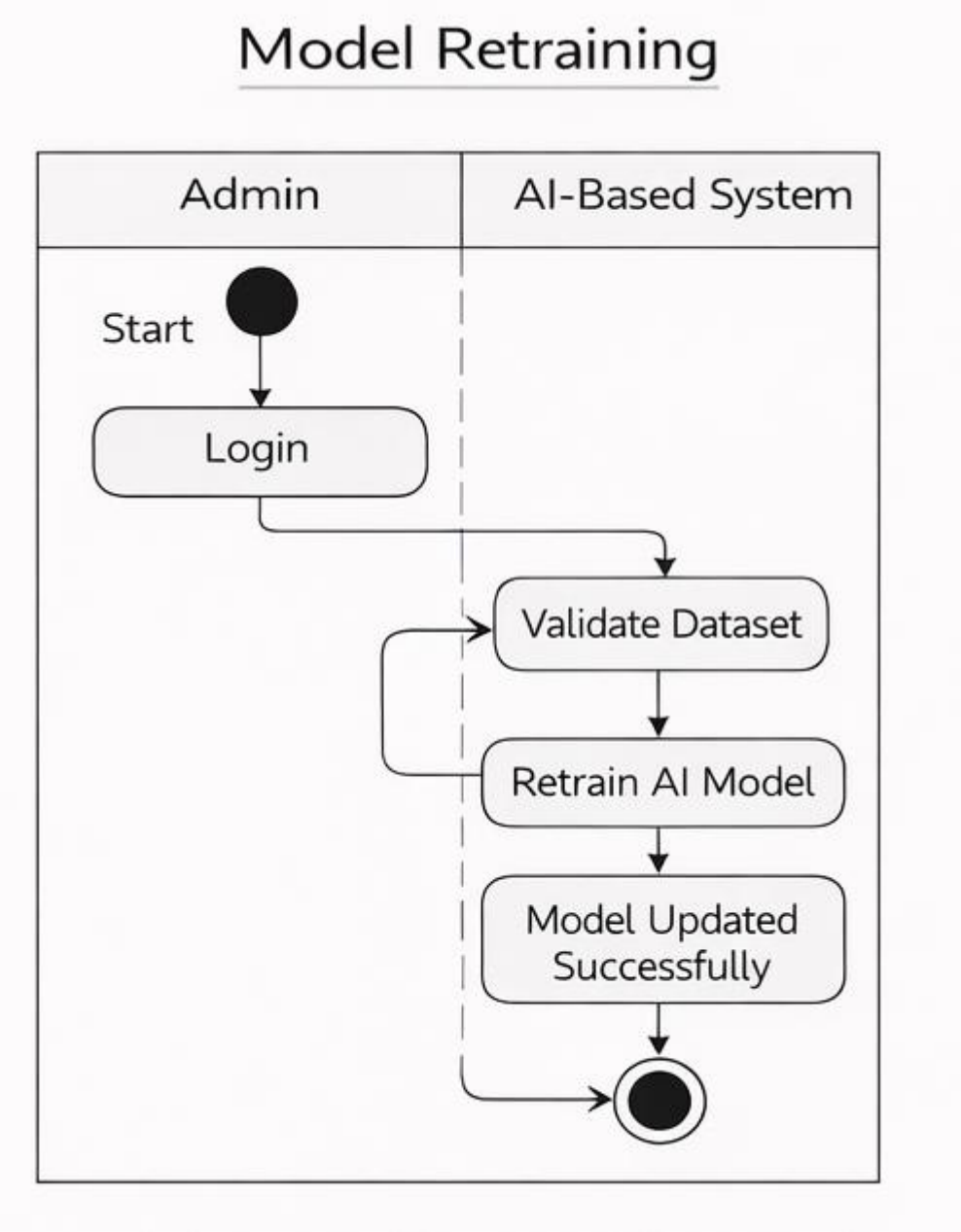
Table 4: The Notation of Activity diagram

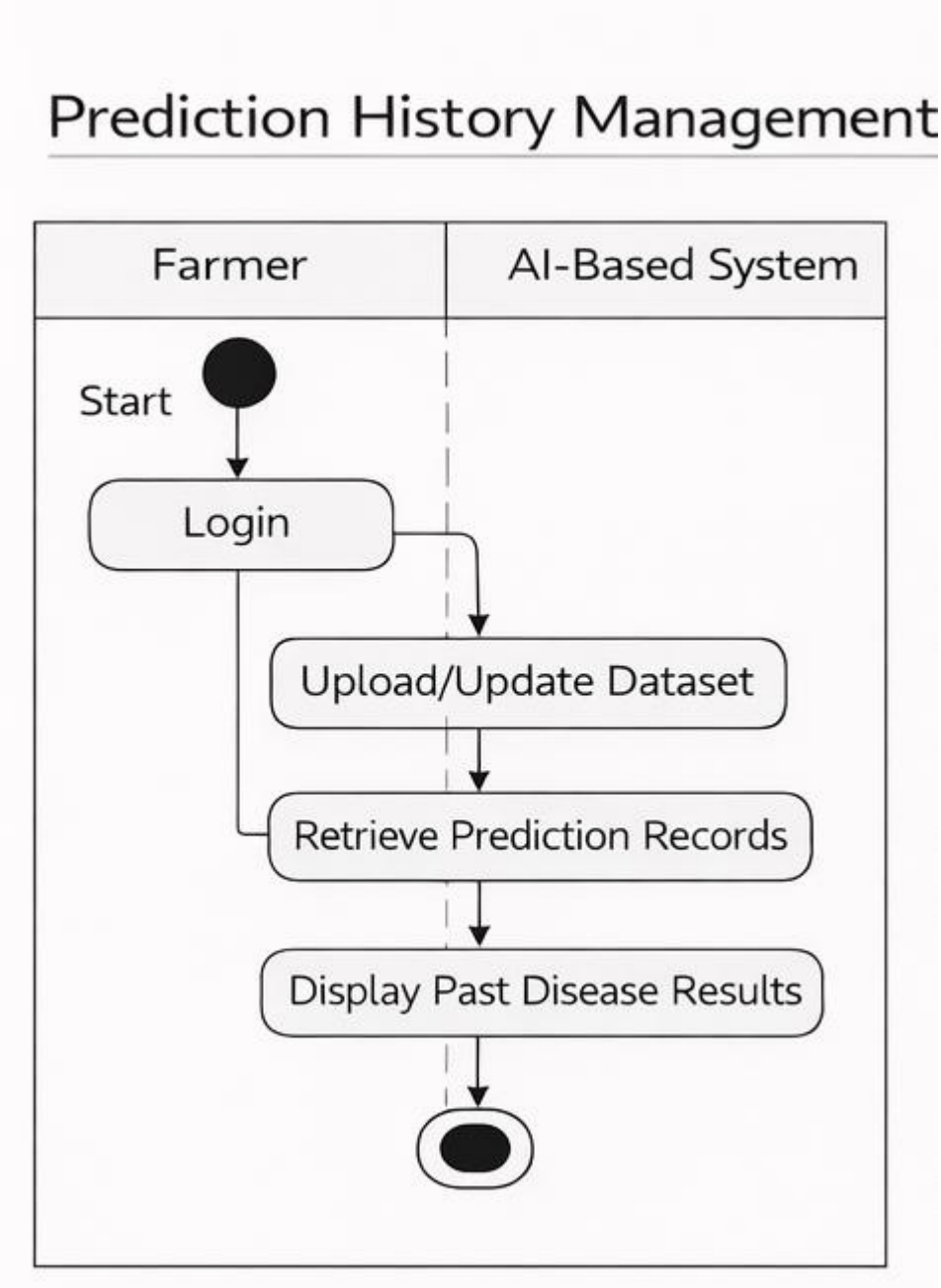
Sr. No	Name	Symbol Description
1.	Start Node	 (Filled black circle)
2.	Action State	 (Rounded rectangle)
3.	Control Flow	 (Arrow line)
4.	Decision Node	 (Diamond shape)
5.	Fork	 (One line splitting into multiple arrows)
6.	Join	 (Multiple lines joining into one line)
7.	End State	 (Black circle inside a white circle)


Crop Disease Detection Process
Farmer
AI-Based Crop Disease Detection System
Start
Login
Upload Crop Image
Receive Crop Image
Image Preprocessing
AI Disease Detection
Disease Detected?
Yes
No
Identify Disease
Show "Healthy Crop"
Generate Confidence Score


Prediction History Management
Farmer
AI-Based System
Start
Login
Upload/Update Dataset
Retrieve Prediction Records
Display Past Disease Results


Model Retraining
Admin
AI-Based System
Start
Login
Validate Dataset
Retrain AI Model
Forum Support


Model Retraining
Admin
AI-Based System
Start
Login
Validate Dataset
Retrain AI Model
Model Updated Successfully

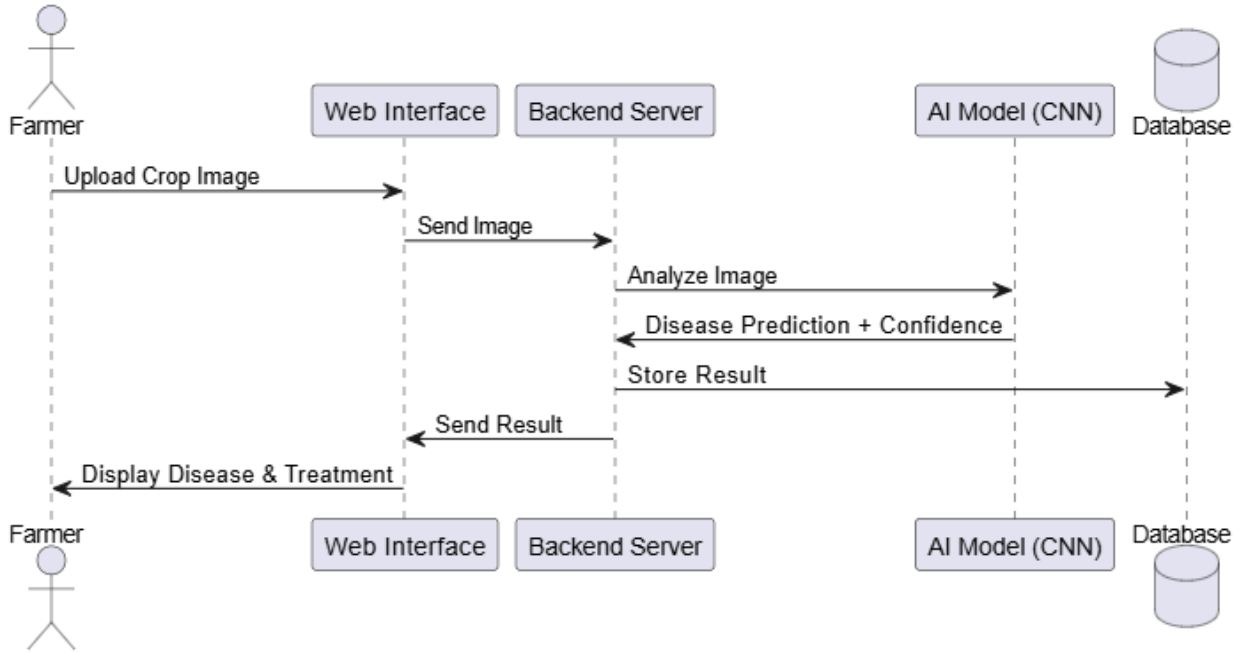


Aligned User Stories:
US003, US004, US005, US006, US007, US008, US011

9.3. Sequence Diagram

A Sequence Diagram is a type of UML interaction diagram that illustrates how objects or system components interact with each other over time. It focuses on the order of messages exchanged between different entities to accomplish a specific functionality or use case. The diagram presents participants as lifelines arranged horizontally, while messages are shown as arrows that indicate the direction and sequence of communication from top to bottom, representing the flow of time (Booch et al., 2005).

Sequence diagrams are particularly useful for modeling dynamic behavior, validating system logic, and understanding how responsibilities are distributed among system components. They help developers and stakeholders visualize system processes, identify dependencies, and ensure that interactions occur in the correct order. By clearly depicting method calls, responses, and control flow, sequence diagrams support effective system design and improve communication among development teams (Sommerville, 2016).



10. References

- Adhikari, B., 2023. *AI-driven agricultural disease prediction using Convolutional Neural * * Networks*. International Journal of Computer Vision and Agriculture, 11(4), pp.45–58.
- Bedi, P. & Dhiman, G., 2022. *Deep Learning techniques for plant disease detection: A survey*. Journal of Applied Artificial Intelligence, 36(2), pp.93–112.
- Ferentinos, K.P., 2018. Deep learning models for plant disease detection and diagnosis. *Computers and Electronics in Agriculture*, 145, pp.311–318.
- Kamilaris, A. & Prenafeta-Boldú, F.X., 2018. *Deep learning in agriculture: State-of-the-art and future trends*. Computers & Electronics in Agriculture, 147, pp.70–90.
- Liu, B., Ding, S. & Xie, C., 2020. *A CNN-based image recognition approach for rapid plant leaf disease classification*. Signal Processing for Agriculture, 18(1), pp.13–25.
- Mohanty, S.P., Hughes, D.P. & Salathé, M., 2016. Using deep learning for image-based plant disease detection. *Frontiers in Plant Science*, 7(1419), pp.1–10.
- Patil, R. & Thorat, S., 2021. *Plant Leaf Disease Detection Using Deep Learning and Image Processing Techniques*. International Research Journal of Engineering and Technology, 8(6), pp.1223–1227.
- PlantVillage Dataset, 2024. *Open access agricultural disease dataset used for model training*. Pennsylvania State University. Available at: https://plantvillage.psu.edu/datasets (Accessed: 28 Nov 2025).
- Rao, A.S. & Thomas, A.R., 2021. *Smart farming: IoT-AI integration for real-time crop monitoring*. Journal of Agricultural Informatics, 12(3), pp.55–68.
- Tumushabe, G., 2022. *Digital farming and AI-based disease control for rural agriculture*. ICT in Agriculture Journal, 19(2), pp.77–95.
- Zhang, J., He, Y. & Qiu, Y., 2020. *Crop disease recognition model based on transfer learning and CNN*. Computers and Electronics in Agriculture, 179, pp.1–10.
- Schwaber, K. and Sutherland, J. (2020) *The Scrum Guide: The Definitive Guide to Scrum – The Rules of the Game*. Available at: https://scrumguides.org (Accessed: 2024).
- Schwaber, K. and Sutherland, J. (2020) *The Scrum Guide: The Definitive Guide to Scrum – The Rules of the Game*. Available at: https://scrumguides.org (Accessed: 2024).
- Sommerville, I., 2016. *Software Engineering*. 10th ed. Boston: Pearson.
- Jacobson, I., Christerson, M., Jonsson, P. and Overgaard, G., 1992. *Object-Oriented Software Engineering: A Use Case Driven Approach*. London: Addison-Wesley.
- Booch, G., Rumbaugh, J. and Jacobson, I., 2005. *The Unified Modeling Language User Guide*. 2nd ed. Boston: Addison-Wesley.
- Ambler, S.W., 2004. *The Object Primer: Agile Model-Driven Development with UML 2*. 3rd ed. Cambridge: Cambridge University Press.
- Booch, G., Rumbaugh, J. and Jacobson, I., 2005. *The Unified Modeling Language User Guide*. 2nd ed. Boston: Addison-Wesley.
- Sommerville, I., 2016. *Software Engineering*. 10th ed. Harlow: Pearson Education Limited.